Empirical Software Engineering (2027) 32:40
https://doi.org/10.1007/s10664-026-10950-z

Vibe Coding: An experiment with test-driven development

Moritz Mock[1] · **Barbara Russo**[1]

Received: 27 October 2025 / Accepted: 4 August 2026
© The Author(s) 2026

Abstract

Context Conversational Large Language Models (CLLMs) can automatically generate code by collaborating with users through natural language. However, poor collaboration can lead to poor quality output.

Objective This exploratory study aims to investigate how humans and CLLMs can collaborate as peers through vibe coding, an approach that integrates principles from prompt engineering, agile design, and human-AI co-creation to enhance collaboration.

Method We designed four interaction models representing different collaboration patterns in the software development process: the solo model (human-only development), the collaborative model (human–CLLM collaboration), the fully automated model (development autonomously performed by a CLLM), and the agentic model (development autonomously performed by the MetaGPT X platform). Based on these models, we implemented corresponding Test-Driven Development (TDD) workflows using structured prompts and Python scripts. We then conducted a controlled pre-experimental study with TDD professionals to compare the solo and collaborative workflows. In addition, we performed repeated exploratory executions of fully automated and agentic workflows on the same development tasks to obtain complementary evidence.

Results Our findings suggest that the choice of interaction model should depend on the development objective. Agentic workflows are best suited for rapid development and functionally correct production code but may introduce additional implementation complexity. However, they may also introduce additional implementation decisions that are not explicitly required by the functional specifications, resulting in untested decision points. In contrast, collaborative workflows produce higher-quality, better-organized test suites.

Conclusions Our work explored how humans can collaborate with CLLMs in generating code in highly intensive and demanding development process. Our results also show that accurately reproducing development process workflows is yet an open problem, also in the new agentic technologies.

Keywords Conversational large language model · Vibe coding · Human-AI collaboration · Prompt engineering · Test driven development

Communicated by: Markus Borg.

Extended author information available on the last page of the article

Published online: 09 October 2026

1 Introduction

Conversational LLMs (CLLMs), such as ChatGPT, are Large Language Models (LLMs) designed to engage in natural, human-like conversations (White et al. 2023). CLLMs have been introduced in software development to streamline and automate various tasks, such as requirement analysis and code generation (White et al. 2024). As such, the interest of academia and industry in these models has increased rapidly (Ebert and Louridas 2023). However, introducing CLLMs in software development comes with a cost. The quality of the answers obtained may not have the desired level (ERCEA 2023; Butler et al. 2025). The characteristics of the selected model, the data used in model pre-training, and the way developers query it may indeed impact the quality of the generated output (Goodfellow et al. 2016; Bishop and Bishop 2023; Liu et al. 2024; Tie et al. 2024). Inadequate queries can, for example, lead to unfaithful responses - called hallucinations, Vaidya and Asif (2023); Maynez et al. (2020) - that developers can directly incorporate into their software without further verification (Fan et al. 2023; Yang et al. 2024). The role of human developers and their interaction with an AI is also a key concern (Butler et al. 2025; Ulfsnes et al. 2024). Although machines are typically used to replace human developers in automating simple repetitive engineering tasks (Butler et al. 2025; Ulfsnes et al. 2024; Vecchio et al. 2024), developers may still fear that their role could lose value.

Recent literature has explored ways to integrate CLLMs into software development. Among the approaches, vibe and agentic coding are the most popular. The former emphasises co-creation, where developers guide CLLM step by step through prompts, while the latter enables CLLM agents to autonomously plan, execute, test, and iterate on development tasks with minimal human intervention, which is typically limited to setting overall goals (Tholander and Jonsson 2026). These represent two different coding philosophies: vibe coding fosters AI-human collaboration and empowers human creativity through prompting, while agentic coding aims at CLLM autonomy and delegation, allowing humans to define coding objectives (Sapkota et al. 2025).

The goal of our work is to *examine how CLLMs can be employed in software development*. To this end, we explored human-CLLM collaboration in multiple settings and levels of autonomy. Our main contributions are then summarised as follows:

- we investigated the participation of CLLMs in software development under varying degrees of autonomy, ranging from vibe coding to agentic coding;
- we designed and integrated human-CLLM interaction models;
- we developed novel prompt patterns to support test and code generation;
- we designed and conducted a controlled pre-experimental study with distributed professional developers on the with Test-Driven Development (TDD) process, Campbell and Stanley (2015);
- we designed and performed an experiment with CLLM-based fully automated and agentic solutions;
- we developed a reproducible toolbox to replicate our experiment with professionals in distributed collaborative environments (*e.g.,* Google Colab).

The remainder of the paper is organised as follows: Section 2 provides background on CLLMs, prompt engineering, vibe and agentic coding, and introduces to TDD. Section 3

presents our methodology for automated TDD with a CLLM, detailing the interaction models we have defined, the prompt design, and the automated vibe coding TDD models. In Section 4, we present the research questions and the empirical study. Followed by Section 5, which presents the results of the empirical study, going over to the discussion and the implications of this work in Section 6. Section 7 reports threats to the validity of our results. Section 8 discusses previous research related to our work. Finally, Section 9 concludes the paper and provides an outlook to future work.

2 Background

In this section, we provide an overview of the relevant background on the major areas of this work: CLLMs, prompt engineering, vibe coding, agentic coding, and TDD.

2.1 Conversational Large Language Models

CLLMs synthesise or generate answers to the questions posed to them (White et al. 2023). CLLMs can also be used for tasks such as reasoning, summarisation, translation, and coding. Current models, such as OpenAI's GPT-4 (Openai 2024), typically rely on the transformer architecture (Vaswani et al. 2017) and are pre-trained (Devlin et al. 2019) on large corpora. Pre-training updates the weights in their neural network architecture through a next-token prediction technique (Ebert and Louridas 2023). To use them for a specific task, the models are typically trained with task data (*i.e.,* fine-tuning), although it is not completely clear whether fine-tuning is always needed (Giagnorio et al. 2025). The pre-trained models can indeed be directly interrogated (*i.e.,* prompting Min et al., 2024). Prompting makes use of the broad range of abilities acquired during pre-training, allowing a model to quickly adapt to a specific task at inference time, guided by natural language instructions or a few examples of the task (Brown et al. 2020). What distinguishes CLLMs from general LLMs is their ability to maintain context across multi-turn conversations, enabling more natural and adaptive interactions. Since the introduction of ChatGPT by OpenAI in November 2022, other CLLMs have been released to the general public, such as Google DeepMind, Antropic, or Meta AI; which is the most preferred is still not clear (Chiang et al. 2024). However, CLLMS also face limitations, including hallucinations, biases inherited from training data, and challenges in explainability. Recent advances, such as instruction tuning, can improve their usability (White et al. 2023, 2024). In this work, we use ChatGPT with an API key to be able to automatically query the CLLM from our scripts. We also use prompt templates to tune the instructions for our specific task, as described in the following.

2.2 Prompt engineering

A prompt is a textual instruction that guides CLLMs in generating appropriate responses. The effectiveness of CLLM's responses also depends on the appropriateness and informativeness of the prompt (White et al. 2023). Fig. 1 illustrates two prompts for the same code-generation task. The prompt on the left specifies only the desired functionality, whereas the prompt on the right additionally provides contextual information, a more precise input specification, and an implementation constraint, *i.e.,* the default line width.

Less informative prompt

Develop a class in Python called `TextFormatter` that takes arbitrary words and horizontally centers them into a line.

More informative prompt

You are a professional developer in a TDD team. Develop a Python class `TextFormatter` that takes a list of words and formats them into strings of a given fixed width (default 80).

Fig. 1 Example of two prompts for the same code-generation task. The prompt on the right provides more contextual information and implementation constraints than the one on the left

On the one hand, prompting CLLMs is fast and easy, requiring little technical expertise compared to approaches such as fine-tuning. On the other hand, understanding the type and amount of information that a prompt should carry for the CLLM to output quality responses (*e.g.,* quality code) is a challenge. Writing prompts becomes an art. To help design prompts, White et al. (2023) introduced the *prompt pattern*, *i.e.,* a solution to rigorously write prompts. Patterns have six components:

- *a name and classification.* The name provides a unique identifier for the pattern that can be referenced in discussions, and the classification groups the pattern with other patterns based on the types of problems they solve;
- *the intent and context.* They capture the problem that the pattern solves and the goals of the pattern;
- *the motivation.* It explains the rationale and importance of the problem that the pattern is solving;
- *the structure and key ideas.* The structure describes the fundamental contextual information that needs to be provided by the LLM to achieve the expected behaviour;
- *example code.* It shows specific implementations of the pattern and discusses them;
- *consequences.* They discuss the pros and cons of using the pattern and discussion of how to adapt the pattern for different situations.

In addition, a catalogue of prompt patterns has recently been proposed (White et al. 2023). The catalogue reports successful approaches for systematically engineering different outputs and interaction goals when working with CLLMs. The patterns and their categories are illustrated in Table 1.

The catalogue has also been further specialised for automating software engineering tasks (the AST catalogue, Table 2) (White et al. 2024). The AST catalogue includes 14 patterns for four software engineering tasks: *Requirements Elicitation, System Design and Simulation, Code Quality, and Refactoring*. No specific category and prompt patterns have been proposed for testing tasks and code generation in testing.

2.3 Vibe Coding and Agentic Coding

Vibe coding and agentic coding follow different philosophies. Vibe coding integrates principles from prompt engineering, agile design, and human-CLLM co-creation to foster the collaboration between human and AI, while abstracting much of the linguistic burden onto the intelligent model. *Vibe coding* uses intuitive, human-CLLM interaction through prompt-

Table 1 Categories and their patterns in the CLLM catalogue of White et al. (2023)

Category	Pattern
Input Semantics	Meta Language Creation
Output Customization	Output Automater
	Persona
	Visualization Generator
	Recipe
	Template
Error Identification	Fact Check List
	Reflection
Prompt Improvement	Question Refinement
	Alternative Approaches
	Cognitive Verifier
	Refusal
	Breaker
Interaction	Flipped Interaction Game Play
	Infinite Generation
	Context Control
	Context Manager

Table 2 Classified prompt pattern for Automated Software engineering Task (AST) by White et al. (2024)

Software Engineering task	Pattern
Requirements Elicitation	Requirements Simulator
	Specification Disambiguation
	Change Request Simulation
System Design and Simulation	API Generation
	API Simulation
	Few-shot Example Generation
	Domain-Specific Language (DSL) Creation
	Architectural Possibilities
Code Quality	Code Clustering
	Intermediate Abstraction
	Principled Code
	Hidden Assumptions
Refactoring	Pseudo-code Refactoring
	Data-guided Refactoring

based, conversational workflows. The term has been popularised by Andrej Karpathy[1] to define how developers describe what they want from a CLLM in natural language prompts, so that the CLLM generates the corresponding code. Agentic coding, instead, aims to minimise the need for continuous human oversight. In agentic coding, AI agents autonomously initiate action, access tools and APIs, retrieve and process external data, and iteratively refine outputs through cycles of self-evaluation, making them suitable for process automation (Sapkota et al. 2025). Agentic coding enables autonomous software development through goal-driven AI agents capable of planning, executing, testing, and iterating tasks with minimal human collaboration and intervention (Sapkota et al. 2025). Examples of tools that enable agentic coding are Google AI, Codex, Anthropic's Claude Code, whereas tools

[1] https://x.com/karpathy/status/1886192184808149383?lang=en

for vibe coding are the CLLMs like OpenAI ChatGPT and Google AI Gemini. If these tools have been typically introduced as plugins for existing development environments, the latest technical advancement is that agentic online development platforms such as Replit[2] and MetaGPT X[3]. MetaGPT X provides multiple agents also actuated by different CLLMs. The platform can implement full-stack applications with collaborating agents that take on the roles and follow predefined workflows - called standard operating procedures (SOP) - that execute the actual development (Hong et al. 2024). SOPs have been introduced to avoid problems causing "unproductive cycles" for multi-agent systems (Li et al. 2023). These problems arise, for example, when cooperating independent agents engage in repetitive, contextless conversations and remain stuck while addressing complex tasks (Talebirad and Nadiri 2023).

2.4 Test Driven Development

TDD is an iterative and incremental development process where tests drive the development of the business logic (Beck 2022). It consists of a sequence of micro-cycles (called iterations) (Fucci et al. 2017), following a predefined sequence of steps: 1) writing a failing test, 2) adding the minimum production code to make the test pass, 3) refactoring the code to improve its design, as illustrated in Fig. 2. The step in which the test code is written is used to define the production code API and expected behaviour. The test is used to drive the external class interface in the intended direction. The focus on creating the minimum code to make the current test suite pass contributes to simpler code. That, together with the safety net created by the regression tests, favours a continuous refactoring process, which has the role of continuously improving the production code's internal design.

Assertion First (Beck 2022) is a TDD pattern that suggests starting to write the test with the assertion. Each assertion should clearly define the input values, the expected output, and a pass-and-fail criterion. After defining the assertion, the rest of the test is defined to create the test scenario for that assertion. The Assertion First pattern allows the developer, when

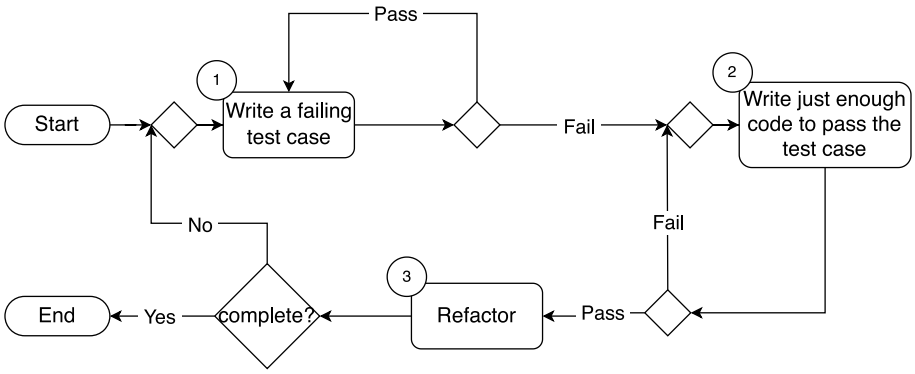


Fig. 2 Overview of the TDD process

[2] https://replit.com

[3] https://mgx.dev

creating the test code, to initially focus on the core of what is being verified, without worrying about the test preconditions.

3 Methodology

For any development process, we propose a four-fold methodology to: (1) design interaction models between humans and CLLMs for software development tasks; (2) define the prompts required to support these interactions; (3) model the resulting development process as a workflow; and (4) evaluate the quality of the generated artifacts and the efficiency of the development processes.

3.1 Interaction Models

In the following, we present the four interaction models considered in this work. Each model is described in terms of who writes the code, how the code is written, and whether the development process is automated.

- *solo-coding* : The development process is performed in the pre-AI manner, where humans write production code and test code by following development practices (*e.g.,* TDD). There is no use of CLLMs and no automation of the process;
- *collaborative* : The human developer and the CLLM collaborate with predefined roles (*e.g.,* the human develops tests and the CLLM develops code) by following development practices (*e.g.,* TDD). This interaction model enforces vibe coding, and the automation supports the collaboration through suitable scripts;
- *fully-automated* : The generation process is carried out entirely by a single CLLM that writes production and test code and the process (*e.g.,* TDD) is fully automated through scripts. This model uses agentic coding, but the human designers still implement the automation of the process, and the human developers trigger the initial step of the development process via prompt;
- *agentic* : The development process is entirely delegated to an agentic platform. An agentic platform is a software framework that coordinates multiple autonomous AI agents, each assigned specialized roles and responsibilities, to collaboratively execute development tasks through predefined workflows. The development process is initially triggered by human developers, but there is no control on the agents neither on the process.

3.2 Prompt Design

A prompt is a natural language input sequence that specifies a task for a language model, often by instantiating a template with input data and leaving one or more answer slots to be completed by the model, Liu et al. (2023).

We define prompts for a development task by leveraging the template catalogue proposed by White et al. (2023) and reported in Table 1. Following Liu et al. (2023), we construct prompts that provide sufficient contextual information and exploit the dialogic capabilities of CLLMs, without attempting to optimise them for maximum model performance, as this lies beyond the scope of this study. Our approach is to design prompts that frame the devel-

opment task as a game involving multiple players, where the objective is to generate code according to a given template. We also allow the task to be iterative so that the code can be refined. Within this setting, the CLLM may act as one or more players in the game. Following the above approach, we can define two prompts: one for test and one for production code. To implement them, we select and combine the following relevant templates from the CLLM catalogue Table 1: *Game Play*, *Persona*, *Template*, *Context Manager*, and *Question Refinement*. The *Game Play* together with the *Persona* pattern allows us to describe the rules (*e.g.,* use *Assertion First*) and the role in the TDD practice (your role is *Tester*); the *Template* pattern is used to layout the output (*e.g.,* provide minimal code using stubs and drivers); the *Context Manager* pattern is employed to control the contextual information in which the CLLM operates (*e.g.,* ignore already implemented test cases). The patterns also include parameters, whose values vary depending on the stage of the development task. The parameters allow using the *Question Refinement* pattern to iteratively update the prompt over the TDD iterations of the task, starting from the first iteration, for which not all parameters are available.

3.3 Workflows and their Automation

We define a workflow as the operationalisation of an interaction model for a specific development process. While an interaction model identifies the actors and how they interact, a workflow specifies the activities they perform, their execution order, the interaction and decision points, the prompts and parameter values exchanged at those points, and the activities automated for execution and data collection. Applying this definition to the interaction models in Section 3.1 results in four workflows:

- *solo-coding* : The workflow is executed as in the pre-AI era. Only human developers execute development tasks. Any automation is limited to collecting process logs;
- *collaborative* : The *solo-coding* workflow is enriched through collaboration between a human and a CLLM via vibe coding. The human typically develops the tests, while the CLLM generates the production code. Assigning the tester role to the CLLM, while the human assumes the developer role, would neither relieve the human from low-level implementation details nor allow them to effectively guide and evaluate the development process. The automation is supported by prompts to query the CLLM and scripts to collect logs and development output at different stages of the development. The human retains control over the quality of the output through monitoring;
- *fully-automated* : The *solo-coding* workflow is fully automated, and the generation of production and test code is delegated to the CLLM. Prompts support the automation thorough scripts that also collect logs and development output at different stages of the development. The human triggers the initial step and retains control over the quality of the output through monitoring;
- *agentic* : The *solo-coding* workflow is delegated to an agentic platform. The human only initiates the workflow execution in the platform according to a selected configuration. The platform then plans, coordinates, and executes the development activities (processes) through its agents and its internal workflows, without human intervention. The human does not supervise or control the quality of the intermediate outputs during execution.

Figure 3 illustrates the overall automation that we designed for the *collaborative* and *fully-automated* workflows. In the *collaborative* workflow, the human-written test code is provided to the general *Runner*, step ①), which triggers the *Collaborative Runner* (step ②). The *Collaborative Runner* calls the *Developer Handler* (step ③), which mimics the developer's activity and wraps the input (error trace, production code, and test code) into the predefined prompt. The prompt is then passed to the *AI Handler* (step ⑥), which is responsible for the API-based interaction with the CLLM (step ⑦). The *Collaborative Runner* updates the prompts and collects execution traces and repetition counts.

In the *fully-automated* workflow, the general *Runner* triggers the *Fully Automated Runner* (step ④), which calls the *Tester Handler* (step ⑤). The *Tester Handler* mimics the tester, embeds the necessary information into the predefined prompt, and passes the prompt to the shared *AI Handler*, which forwards it to the CLLM (steps ⑥ and ⑦). The generated test code is returned through the *Fully Automated Runner* to the general *Runner*, which then passes it to the *Collaborative Runner* for production-code generation. Step ⑥ also includes the shared *Log Collector*, which collects the <error trace> for both workflows. In the *solo-coding* workflow, only logging is performed.

3.4 Evaluation Framework

The data collected in this study comes from two complementary sources. First, we conducted a controlled pre-experimental study involving professional developers to evaluate the *solo-coding* and *collaborative* workflows. Second, we replicate the study with *fully-automated* and *agentic* workflows to provide exploratory evidence on autonomous software generation. To compare the output resulting from the four models of interaction, we developed an evaluation framework along three complementary dimensions: production code quality, test code quality, and development process efficiency. The framework defines (i) the quality attributes of interest, (ii) the metrics used to operationalize each attribute, and (iii) the statistical procedure adopted to compare the workflows. The corresponding metrics are reported in Table 4.

The framework evaluates the quality of the generated production code through complementary quality attributes. First, it assesses syntactic similarity to determine whether different workflows produce comparable implementations. It then evaluates functional cor-

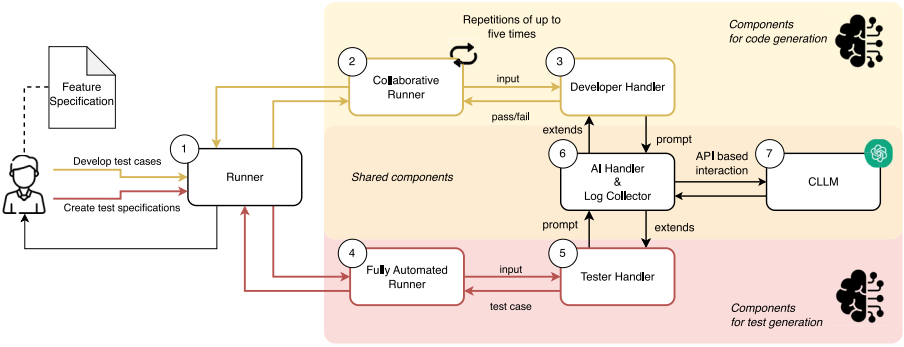


Fig. 3 Overview of the toolset's architecture

rectness and structural complexity, two quality sub-attributes commonly considered when assessing generated code (Truong et al. 2026).

The framework also evaluates the quality of the generated test suites. Specifically, it considers structural complexity and test effectiveness, measured through the coverage achieved by the generated test suites on the corresponding generated production code. These quality attributes have been identified as key indicators of generated test quality (Tran et al. 2021). Finally, the framework evaluates the efficiency of the development process through metrics describing the effort required to complete the task.

The collection of the metrics' values is performed through experiments. Thus, for each metric, we obtain four sets of values each for one workflow. We then apply statistical tests to capture difference among metrics and their related quality attributes across the four workflows. We first apply the Kruskal-Wallis (KW) H test (Kruskal and Wallis 1952) to determine whether at least one workflow differs significantly from the others. We use H as an initial screening step before performing pairwise comparisons. KW H test is a non-parametric rank-based test that checks whether different groups originate from the same distribution. The test is significant if at least one group does not come from the same distribution. The test is suitable in our settings as it does not assume a specific underlying distribution or require a particular sample size. We report the H value, the p-value, and the effect size ϵ^2 for each metric to assess the significance and the magnitude of the observed differences. The effect size ranges from 0 to 1, with values of 0.01-0.08 indicating a small effect, 0.08-0.26 a medium effect, and values of at least 0.26 a large effect, Mangiafico (2025). To compare pairwise the four sets of values of a metric, we use one-tailed Mann-Whitney (MW) U test (Mann and Whitney 1947), reporting the U value, p-value, and Cliff's delta effect size to assess the significance and the magnitude of the observed differences. Cliff's delta ranges from -1 to 1, with 0 indicating stochastic equality of the two groups. 1 indicates that one group shows complete stochastic dominance over the other group, and a value of –1 indicates the complete stochastic domination of the other group. The values of the effect size 0.11-0.28 indicate a small effect, 0.28-0.43 a medium effect, and values of at least 0.43 a large effect, Mangiafico (2025). For each pairwise MW U test, we use $\alpha = 0.05$ as the conventional significance threshold in software engineering, Kitchenham et al. (2017). We report the p-values and effect sizes to enable readers to interpret the results under alternative significance thresholds, in line with the recommendations of Wasserstein et al. (2019). In addition, to account for multiple pairwise comparisons on the same metric, we also apply the Holm–Bonferroni (HB) correction, Holm (1979). Unlike the standard Bonferroni correction, HB controls the error rate of a family of tests, making it particularly suitable for studies with a limited sample size. The procedure first orders the m p-values in ascending order. Starting from the smallest p-value, each is compared against the progressively less stringent significance threshold $\alpha/(m - i + 1)$, where i denotes its rank in the ordered list and α is the single-test significance threshold. The procedure stops as soon as a comparison is not significant; all subsequent hypotheses are then retained. In our study, we perform six pairwise comparisons for each metric (except in one case, where only three comparisons are possible because measurements could not be obtained for one workflow). Using $\alpha = 0.05$, the corresponding HB significance thresholds are illustrated in Table 3.

To complement the quantitative analysis, we inspected the generated artifacts and the execution logs. We also collected feedback from the human participants to gain deeper insights into their decisions and their experience during the experiment.

Table 3 HB thresholds for six and three tests with $\alpha = 0.05$

Rank	six tests	three tests
1	0.0083	
2	0.010	
3	0.0125	
4 (1)	0.0167	0.0167
5 (2)	0.0250	0.0250
6 (3)	0.050	0.050

4 Empirical Study

To illustrate the feasibility of our approach, we apply our methodology to the TDD process through an empirical study. We chose TDD as highly intensive and demanding development process. The major strength of TDD lies in its ability to deliver high-quality code through the granularity and uniformity of development, Fucci et al. (2017). To be effective, TDD developers must have a strong command of the practice and experience in development, Causevic et al. (2011). In this section, we describe the empirical goal and the research questions, the design of the experiments, and the automation of the TDD workflows according to the four interaction models.

4.1 Empirical Goal and Research Questions

The goal of our work is to examine *how CLLMs can be employed in software development*. We adopt the Goal-Question-Metric (GQM) paradigm (Caldiera and Rombach 1994) to systematically derive, from our research goal, the empirical goal (G), the research questions (Q), their associated sub-questions, and the corresponding metrics (M). The metrics are then used in the evaluation framework we introduced in Section 3.4.

Our main empirical goal is then:

G: Analyse code generation to compare human-CLLM interaction models with respect to code quality and development process efficiency, from the viewpoint of developers, in the context of TDD.

We follow our evaluation framework in Section 3.4 to derive the RQs and the corresponding metrics as in the following.

RQ1. *How does the quality of the production code created with different interaction models vary?* To investigate this aspect, we first analyse the syntactic similarity among the production code obtained with the different workflows and then, evaluate its quality along two complementary dimensions: functional correctness, and code complexity.

RQ1.1. *How does the syntactic similarity of the production code created with different interaction models vary?* Syntactic similarity represents the degree to which different interaction models generate independent solutions. We use the Longest Common Subsequence (LCS) and Levenshtein Edit Distance (LED) as in recent literature, Ouyang et al. (2025).

RQ1.2. *How does the functional correctness of the production code created with different interaction models vary?* Second, we focus on functional correctness, as fundamental software quality sub-attribute (ISO25010 2024). To evaluate correctness, we first implemented a *baseline test suite* from the feature specification in Fig. 4 by applying category-partition testing (Ostrand and Balcer 1988). Category-partition testing begins by identifying the input parameters of the function under test. The value domain of each parameter is then

The goal of this experiment is to develop in Python the following feature:

Develop a class TextFormatter that takes arbitrary words and horizontally centres them into a line.

The class TextFormatter shall have three functions:

- setLineWidth sets the length of the line.
- The second function receives a single word and returns the word in the centre of the line. *First task.*
- The third function receives two words and centres the two words in the line. *Second task.*

To develop it you will use *Test Driven Development with assertion first.*

Fig. 4 Tasks' specification

partitioned into categories, where each category groups values expected to trigger the same behaviour of the function. The resulting categories are then refined to eliminate redundant input combinations, and representative values are selected from each category. Using this process, we derived a set of test cases for each of the two development tasks and implemented the corresponding test methods and assertions. Listings 1 and 2 illustrate the code for the baseline tests for the first task and the second task, respectively.

Listing 1 Method and its Assertions for the Task 1.

```
def test_centerString(self):
  self.assertEqual(centerString("word", 8), "␣␣word␣␣")
  self.assertEqual(centerString("hello", 8), "␣hello␣␣")
  self.assertEqual(centerString("hello", 5), "hello")
  self.assertEqual(centerString("hello", 0), "")
  self.assertEqual(centerString("", 3), "␣␣␣")
```

Listing 2 Method and its Assertions for Task 2.

```
def test_centerTwoStrings(self):
  self.assertEqual(centerTwoStrings("a", "b", 5), "␣a␣b␣")
  self.assertEqual(centerTwoStrings("a", "b", 6), "␣a␣b␣␣")
  self.assertEqual(centerTwoStrings("a", "b", 2), "ab")
  self.assertEqual(centerTwoStrings("a", "b", 0), "")
  self.assertEqual(centerTwoStrings("", "b", 4), "␣b␣")
  self.assertEqual(centerTwoStrings("a", "", 4), "␣a␣␣")
  self.assertEqual(centerTwoStrings("", "", 4), "␣␣␣␣")
```

We then evaluate the success rate of the baseline tests executed on the production code developed or generated in the four workflows. Specifically, we measure the Test Pass Rate (TPR), defined as the ratio of passed baseline tests to the total number of baseline tests (Ostrand and Balcer 1988).

RQ1.3. *How does the complexity of the production code created with different interaction models vary?* In addition to correctness, we analyse code complexity to capture internal code characteristics that impact maintainability as sub-attribute of quality (ISO25010 2024). Specifically, we focus on code structural complexity and code coverage achieved by the baseline tests. To assess structural complexity, we use McCabe complexity (MCC) (Mccabe 1976), a widely used metric that quantifies decision flows in the code (Meneely 2016). For code coverage against baseline tests, we use statement coverage SCB and branch coverage BCB .

Since TDD systematically produces test code throughout the development process, evaluating the quality of the generated tests is important. We therefore compare the test suites generated in the four workflows against the baseline test suite and evaluate the coverage achieved by each generated test suite on its corresponding generated production code.

RQ2. *How does the quality of the tests created with different interaction models vary?* To identify the metrics to use, we refer to the quality model defined in Tran et al. (2021) and to the quality sub-attributes structural complexity and code coverage, which are relevant for practice, Tran et al. (2025).

RQ2.1. *How does the structural complexity of tests created with different interaction models conform to that of the baseline tests?* To answer this question, we use the number of test methods (TM) and the average number of assertions per method (TAM).

RQ2.2. *How does the coverage of the code created with different interaction models vary?* To answer this question, we execute the tests created with the different interaction models on the corresponding code and measure statement (SC) and branch coverage (BC).

With the last research question, we shift our focus from the product to the process.

RQ3. *How does the efficiency of the TDD process vary between different interaction models?* We measure process efficiency as the effort required to complete a task, measured through the number of iterations (# of Iterations) and the completion time, in line with definitions of efficiency as performance relative to resource usage (ISO25010 2024) and studies on performance in solving program-comprehension tasks (Astromskis et al. 2017; Siegmund et al. 2014). Specifically, we employ the number of TDD iterations (# of Iterations) and completion time (Time), which is both self-reported by participants in the human-centred models and collected from the runtime in the automated models.

Table 4 summarises the mapping between the three research questions and the metrics used to answer them and define our evaluation framework described in Section 3.4. Each metric is presented with its acronym, name and description. The table also reports CLOC and TLOC for completeness and to provide an indication of the size of the generated artifacts.

4.2 Design of the Experiment

Firstly, we conducted a *preliminary experiment* with five professional developers (Mock et al. 2025) to assess the feasibility of the study and collect feedback on the experimental protocol. Based on the insights gained from the preliminary experiment, for the study reported in this paper, we extended the experiment with an additional 11 professionals, using the same toolset and experimental procedure and task. Although the total sample size falls within the typical range for empirical studies involving professional software engineers (15–25 participants Kitchenham and Madeyski, 2024), the results remain subject to the limitations associated with a relatively small sample of human participants. To mitigate potential threats arising from this limitation, we adopted a threefold strategy. First, we followed a pre-experimental design (Campbell and Stanley 2015). Second, we employed non-parametric statistical tests and controlled statistical power through multi test significance with significance threshold correction (Section 3.4). Third, we complemented the statistical analysis with a careful inspection of the data to account for potential small-sample effects. Nevertheless, we acknowledge that the limited sample size may reduce the statistical power of the comparisons involving the human participants. We discuss this limitation and its implications in the Threats to Validity (see Section 7). We executed the *fully-automated* and

Table 4 Metrics per research question

RQ	Acronym	Name	Definition
	CLOC	Code LOC	Number of lines in the production code, empty lines and comments were not counted, positive integer.
	TLOC	Test LOC	Number of lines in the test code, empty lines and comments were not counted, positive integer.
RQ1	LCS	Longest Common Subsequence	Length of the longest common subsequence between two sequences; positive integer.
	LED	Levenshtein Edit Distance	Minimum number of single-token edits (insertions, deletions, or substitutions) required to change one text into the other; positive integer. Levenshtein (1966).
	TPR	Test Pass Rate	Proportion of baseline test cases that pass on the production code over the total number of baseline tests; percentage from 0 to 100.
	MCC	McCabe Complexity	Number of code branches plus one; an integer greater than one. Mccabe (1976)
	SCB	Statement coverage with respect to baseline tests	Proportion of executable statements in the production code that are covered by the baseline tests (Zhu et al. 1997); percentage from 0 to 100.
	BCB	Branch coverage with respect to baseline tests	Proportion of executable branches in the production code covered by the baseline tests (Zhu et al. 1997); percentage from 0 to 100.
RQ2	TAM	Test assertions per method	Average number of assertions per test method; positive real number.
	TM	Number of test methods	Number of test methods; positive integer.
	SC	Statement coverage of the created tests	Proportion of executable statements in the production code covered by tests created in the workflow; (Zhu et al. 1997); percentage from 0 to 100.
	BC	Branch coverage of the created tests	Proportion of executable branches in the production code covered by tests created in the workflow; (Zhu et al. 1997); percentage from 0 to 100.
RQ3	# of Iterations	Number of iterations	Number of TDD iterations recorded by the toolset during the experiment; positive integer.
	Time	Time	Time required to complete the task (in seconds), self-reported by participants in *solo-coding* and *collaborative* workflows, and collected from the runtime in *fully-automated* and *agentic* workflows; positive integer.

agentic workflows on the same development tasks. For the collaborative and fully automated workflows, we used OpenAI's *GPT-3.5 Turbo* through its API.

In all cases, we selected the same two development tasks that did not require understanding complex APIs or class structures. Fig. 4 reports them.

4.2.1 Experiment with Professionals

We applied the Static-Group Comparison, as pre-experimental design with control group (Campbell and Stanley 2015; Wohlin et al. 2012), with the purpose of establishing the effect of a treatment X, Fig. 5. Treatment *X* in our experiment is the application of

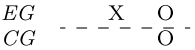


Fig. 5 The Static-Group Comparison with control group: *EG* = experimental group and *CG* = control group, *X* = treatment, and *O* = ex-post observation

the *collaborative* workflow to the TDD process. The design then foresees two groups: the Experimental Group (*EG*) and the Control Group (*CG*), where the EG follows *collaborative* model and the CG the *solo-coding* one. Human participants in the *collaborative* and *solo-coding* groups are randomly assigned and given the same tasks. The metrics are collected only once during the experiment and analysed after the experiment (*O*) to determine the effects of treatment (*X*).

At the end of the experiment, participants also completed a post-experiment questionnaire to assess the perceived difficulty of the task and their overall experience with the development process.

Participants *In the experiments, we only employed IT professionals. Employing professionals in software engineering experiments presents both advantages and drawbacks (Romano et al. 2025; Salman et al. 2015). On the positive side, their participation yields more realistic results, as their behaviour more closely mirrors that of developers in industry. Their experience helps make results more connected to the actual practice. On the negative side, their behaviour is less predictable because they have different backgrounds and habits, so results may be harder to control. It is also difficult to get them involved, as recruiting them requires more time and effort, and their busy schedules often limit their participation (citealtRomanoEtAl2025). The typical sample size of studies involving professionals ranges from 15 to 25 participants (Kitchenham and Madeyski 2024). With 16 professionals, our study falls well within this range. The participants were IT professionals from Italy, Denmark, and Brazil, with experience in TDD and Python, as illustrated in Table 5. All participants were required to complete the same development tasks. They were introduced to the experiment by one of the authors.*

Study Protocol *We carried out the experiment in two ways: remote, using a call and screen sharing and in-person. In both cases, we used Google Colab*[4] *as the development environment, i.e., a cloud-based service that provides computing resources through a Jupyter notebook interface. Using this tool, we were able to control dependencies, ensure the same Python interpreter version, and create a script to collect data during the experiment. Before the experiment, we obtained the consent of the participants: they were informed of their voluntary participation and their right to withdraw at any time without providing a reason. The participants were introduced to the tasks and Google Colab. During the experiment, participants were allowed to search the Internet. However, they were prohibited from searching for complete solutions or independently querying any CLLM, e.g., ChatGPT. This restriction was enforced by monitoring participants via screen sharing or direct observation in the in-person setting. Although this allowed us to verify that participants did not use external AI tools, we cannot completely rule out the possibility that someone may have used an AI tool, e.g., on a second screen. The participants were given 40 minutes to complete the tasks. During the experiment, one of the authors was available to assist the participants with environmental issues or task clarification. No assistance was provided on how to solve the tasks. Participants who interact with the CLLM were not allowed to modify the prompts. After completing the tasks, participants were asked to submit their code and complete a feedback questionnaire, which can be found in the replication package (Mock and Russo 2025).*

[4] https://colab.research.google.com/

Table 5 Demographics of the participants

ID	TDD experience	Python experience	Role
solo-coding participants			
P1	1-3 years	>3 years	Software developer
P2	1-3 years	<1 year	Software developer
P3	<1 year	>3 years	Security software engineer
P4	<1 year	<1 year	DevOps engineer
P5	1-3 years	>3 years	Software developer
P6	<1 year	<1 year	Software developer
P7	1-3 years	>3 years	DevOps engineer
P8	<1 year	<1 year	Software developer
P9	<1 year	1-3 years	Software developer
collaborative participants			
P10	>3 years	<1 year	Software developer
P11	1-3 years	>3 years	Data scientist
P12	<1 year	<1 year	Software engineer
P13	<1 year	<1 year	Software developer
P14	<1 year	<1 year	Technical leader
P15	>3 years	>3 years	Software developer
P16	<1 year	1-3 years	Software developer

4.2.2 Experiment with Agents

For the *agentic* workflow, we selected MetaGPT X (MGX[5]) as a representative for agentic platforms. The platform has two operation modes: Engineer and Team. To resemble our experiment the most, we chose the Engineer mode (single developer). In repeated individual runs, we configured the platform to use one of two models: OpenAI ChatGPT 5[6]and Anthropic Claude Sonnet 4[7]. ChatGPT 5 is the most similar to the model we used in the experiment, while Claude Sonnet 4 is one of the most promising and suitable models in our context (Palla and Slaby 2025; Nascimento et al. 2025). We collected the generated code and computed the same metrics as for the participants. However, we could not collect the number of iterations because the MetaGPT X operates with standard operating procedures (SOPs), which do not support a TDD-like interaction with the user; therefore, the solution was provided in a single response. For the *agentic* workflow we have defined two prompt scenarios: 1) we provided a minimal prompt and 2) we provided a more detailed description of the prompt, like the one used for the *fully-automated* workflow.

In Table 6, the mean value of each metric (see Table 4) is computed for each model and prompt scenario. We observe little difference between the two CLLMs. However, between the two prompt scenarios, minor variations can be observed for some metrics, *e.g.,* MCC , TAM , and TM . Additionally, CLOC and TLOC vary across the prompt scenarios. Overall, these observations mainly relate to the structure of the generated code rather than its functionality. Interestingly, most coverage metrics (SCB , BCB , and SC) remain similar across the four different settings, suggesting that neither the choice of model nor the prompting scenario substantially affects the overall testability of the generated code or the effectiveness of the generated test suites. Given the minor differences in MCC , TAM , and TM , we decided to merge all *agentic* workflow executions and analyse them together rather than individually. Future work is needed to investigate the performance differences of the *agentic* workflow under different prompting settings. Therefore, this part of the study should be considered exploratory.

For the *fully-automated* workflow we created an automated process as described in Section 4.3 and Fig. 6. The workflow executed in a CLI setting, instructed with OpenAI's *GPT-3.5 Turbo* through its API. It was configured with the prompts listed in Table 7; the prompts and their iterative refinements are embedded in the toolset. During execution, the workflow automatically recorded all logs, including the TDD iterations and their intermediate solutions. Execution time was measured using the terminal command `time`, which was prefixed to the CLI execution command. All experiments were conducted on a macOS host machine. As with the *agentic* workflow, we collected the generated code and computed the same evaluation metrics as those used for the participant submissions. The workflow was executed multiple times with the same settings.

[5] MGX was used through the web interface[3].

[6] https://openai.com/index/introducing-gpt-5/

[7] https://www.anthropic.com/news/claude-4

Table 6 Mean metric values for the *agentic* runs, grouped by model and prompt scenario. CG = ChatGPT 5, CS = Claude Sonnet 4

	Production code					Test code					Process	
	CLOC	TPR (%)	MCC	SCB (%)	BCB (%)	TLOC	TAM (Ass.)	TM	SC (%)	BC (%)	Iter.	Time (sec)
CG1	53.67	94.33	15.00	63.33	60.67	41.33	7.48	3.67	97.00	92.33	-	120
CG2	15.67	92.00	9.67	61.00	67.33	22.33	1.00	3.00	85.67	50.00	-	60
CS1	53.00	97.33	13.67	64.33	67.00	35.33	5.77	7.00	83.33	75.33	-	60
CS2	21.00	94.67	8.33	71.00	66.33	25.33	2.00	4.33	85.67	50.00	-	60

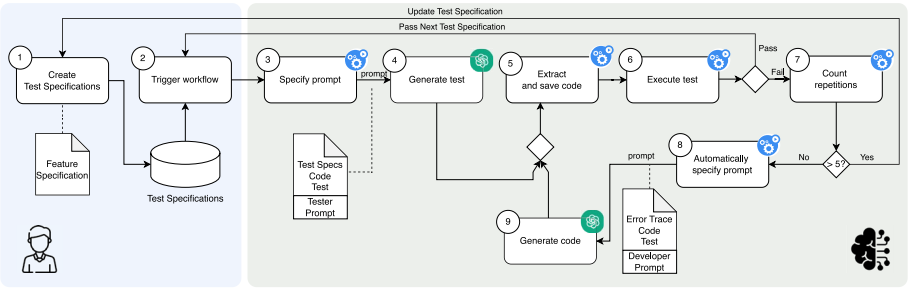


Fig. 6 *fully-automated* workflow

4.3 Implementing the TDD Workflows

In this section, we define the *fully-automated*, *collaborative*, and *agentic* TDD workflows (Figs. 6, 7, and 8), corresponding to the *fully-automated*, *collaborative,* and *agentic* interaction models. In the *fully-automated* and *collaborative* workflows , activities marked with the OpenAI symbol are performed by the CLLM, while those with the gear symbol are executed by our scripts; all remaining activities are carried out by humans. The note symbols indicate the prompts used to query the model, as described in Table 7. In these two workflows, an iteration is the execution of the workflow from the creation of a single assertion until the generated code passes it. A repetition, instead, refers to a single loop cycle in which the workflow is executed for a specific assertion to verify whether the generated code satisfies it. The two interaction workflows only differ in the way the tests are created. The *fully-automated* workflow is initiated by humans with a set of test specifications, ①[8]. The human then selects one test specification and triggers the workflow ②, one for the first iteration and the other for subsequent iterations, as in Table 7, ③. Afterwards, the CLLM is prompted for the generation of the test code ④. Then, our toolset automatically executes the test against any available production code and collects its execution traces, ⑤ and ⑥.

From this point onward, the two TDD workflows of the two interaction models (*fully-automated* and *collaborative*) perform the same activities in a loop, where the CLLM acts as a developer and generates the production code. Whenever the tests fail – for example, during the first iteration when no production code exists yet - the loop (activities ⑦ to ⑨ in Fig. 6 and ④ to ⑦ in Fig. 7) is triggered. The same prompt is automatically reissued up to five times within the same interaction session, using modified prompts from the second iteration onwards Table 7. If the CLLM is still unable to generate production code that satisfies the test code, the loop is interrupted and execution is redirected. In this case, in the *collaborative* mode, where humans act as testers, we allow them to refactor the generated code at their convenience, following TDD. In the *fully-automated* model, we let the CLLM handle the refactoring autonomously by simply instructing it to follow the TDD practice. From the second iteration onward, the entire existing test and production code is passed into the prompts ④ and ⑨, and ⑧ in Figs. 6 and 7, respectively.

[8] It is worth noting that we do not automate the generation of the test specifications or any other natural language artefact, as we want to keep the focus on the generation of code.

Table 7 Prompts designed for the collaborative, fully-automated, and agentic interaction workflows of the TDD process

Collaborative

The human derives the test case specifications from the feature. At each iteration, the human develops a test code - *i.e.,* one assertion and the surrounding test code that ensures its execution - for a test case specification. At the end of each iteration, the test code is executed and a new error trace is produced.

First iteration; **CLLM role: developer**

You are part of a TDD team. Your role is developer. Based on the information provided below, write the minimal production code necessary to ensure that the test case no longer fails. Built-in functions are not allowed.

<error trace, test code>

Second and further iterations; **CLLM role: developer**

You are part of a TDD team. Your role is developer. Based on the information provided below, write the minimal production code necessary to ensure that the test case no longer fails. Built-in functions are not allowed.

<error trace, test code, existing test code, existing production code>

Fully automated

At each iteration, the CLLM is given one baseline test case specification. The CLLM develops the test and the production code. At the end of each iteration, the test code is executed, and a new error trace is produced.

First iteration; **CLLM role: tester**

You are part of a TDD team. Your role is tester. Based on the information provided below, write test code using the 'unittest' library. Use Assertion-First in TDD. Focus only on the test case specification described below. Do not implement solutions for other test cases.

<test case specification>

CLLM role: developer

You are part of a TDD team. Your role is developer. Based on the information provided below, write the minimal production code necessary to ensure that the test case no longer fails. The code must be written in the same file of the test. Built-in functions are not allowed.

<test code, error trace>

Second and further iterations; **CLLM role: tester**

You are part of a TDD team. Your role is tester. Based on the information provided below, write test code using 'unittest' library. Use Assertion-First in TDD. Keep all existing tests intact and add one new test case. Focus only on the test case specification described below. Do not implement solutions for other test cases.

<test case specification, existing production code, existing test code>

CLLM role: developer

You are part of a TDD team. Your role is developer. Based on the information provided below, write the minimal production code necessary to ensure that the test case no longer fails. The code must be written in the same file of the test. Built-in functions are not allowed.

<test code, error trace, existing test code, existing production code>

Agentic

The agent is provided with an initial prompt. We experimented with two different prompts.

Prompt 1

Develop the following feature:

<task specification>

To develop it you will use TDD with assertion first.

Prompt 2

You are part of a TDD team. Based on the information provided below, write test code using 'unittest' library. Use Assertion-First in TDD The code must be written in the same file of the test. Focus only on the test case specification described below. Do not implement solutions for other test cases.

<task specification>

Templates: Game Play, Persona, Template, Context Manager, Question Refinement

Fig. 7 *collaborative* workflow

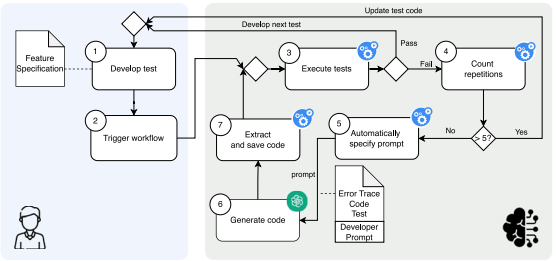
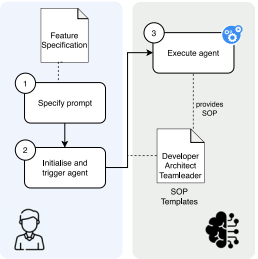


Fig. 8 *agentic* workflow



Iteration	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Assertions	1(+1)	4(+3)	4(0)	4(0)	3(−1)	5(+2)	4(−1)	5(+1)	5(0)	5(0)	5(0)	9(+4)	8(−1)	9(+1)
Test func.	1(+1)	2(+1)	2(0)	2(0)	2(0)	2(0)	2(0)	2(0)	2(0)	2(0)	2(0)	3(+1)	3(0)	3(0)
Tries	1	5	5	5	1	5	1	5	5	3	5	5	5	5
Result	OK	FAIL	FAIL	FAIL	OK	FAIL	OK	FAIL	FAIL	FAIL	FAIL	FAIL	OK	FAIL

Fig. 9 Example of log data in *collaborative* process

In the *agentic* workflow, the human spcifies the prompt from the feature specification, 1, initialieses and triggers the agent, 2, and leaves the entire development process to the agent, which executes it using its SOP templates, 3, as illustrated in Fig. 8.

The execution data we obtained from the *fully-automated* and *collaborative* workflows are stored in logs. Fig. 9 shows the log for participant P15 with *collaborative* workflow and 14 iterations (columns). The first row describes the cumulative number of assertions developed by the participant. The second row reports the cumulative number of test functions. The third row counts the repetitions and the last row reports the output of the test execution. In the first iteration, the CLLM generates code that passes the test method developed by the participant ("OK"). In the second iteration, the participant develops a new test function, with three assertions. For this test case, the CLLM was unable to generate production code that passes the tests in five attempts. Thus, the participant iterated the generation for two further times (iterations 3 and 4), before the participant modified the test cases in iteration 5.

4.3.1 Prompts

Table 7 illustrates the prompts we designed for the workflows *collaborative* and *fully-automated* . Colours identify the relevant templates as listed in Table 1, while angular brackets <...> identify the parameters. For the *collaborative* and *fully-automated* models, the table also shows two types of prompts: the one used in the first iteration and the one used in subsequent iterations of the TDD process.

For example, in the first iteration of the *collaborative* workflow, no production code is yet available; therefore, the prompt parameters consist only of <test code> and <error trace>.

Listing 3 Example of test code and error trace with one assertion for the feature that develops the class TextFormatter.

```
# Test code:
import unittest
class TestTextFormatter(unittest.TestCase):
    def test_center_word(self):
        f = TextFormatter()
        f.setLineWidth(10)
        centered_string = f.center('word')
        self.assertEqual(centered_string,'___word___')
if __name__ == '__main__':
    unittest.main()
# Error Trace
Traceback (most recent call last):
  File "$<$path$>$", line 18, in test_center_word
    self.assertEqual(centered_string,'___word___')
AssertionError: None != '___word___'
```

Listing 4 Example of generated code with the free online version of ChatGPT 3.5-turbo.

```
class TextFormatter:
    def setLineWidth(self, width):
        pass
    def center(self, string):
        if len(string) $>$= 10:
            return string
        else:
            num_spaces = (10 - len(string)) // 2
            return ' ' * num_spaces + string + ' ' * num_spaces
```

An example of such input is shown in Listing . From the CLLM's response, the production code is extracted; an example is provided in Listing .

In subsequent iterations, the production code is included in the prompt (see Table 7) together with the <test code> and <error trace>. Additionally, prompts incorporate the production and test code generated in previous iterations. This enables the incremental generation of both code and tests throughout the process. Each iteration is executed as a new interaction session to prevent the CLLM from being biased by prior iterations.

For the *fully-automated* model, at each iteration, the CLLM is given in sequence one of the following test specifications:

- line width=10 and word="word",
- line width=10 and word="hello",
- line width=10 and words=["foo", "bar"]

For the *agentic* workflow, Table 7 illustrates the initial prompts each triggering the entire development task. No other prompts have been used.

5 Results

From the RQs, we derived the metrics of our evaluation framework Table 4. We then applied the statistical analysis defined by the framework, beginning with the H test on all metrics. Table 8 shows that H is statistically significant for CLOC, TPR , MCC , BCB , SC , BC ,

Table 8 Kruskal-Wallis H test across models. The asterisk (*) denote statistical significance at $\alpha = 0.05$; effect size computed with ϵ^2. The comparison for # of Iterations excludes the *agentic* workflow

	Metric	H	Effect Size	p-value
	CLOC	12.27	0.34	6.52e-03*
	TLOC	6.22	0.12	1.01e-01
RQ1	TPR	16.98	0.52	7.12e-04*
	MCC	16.59	0.5	8.58e-04*
	SCB	4.88	0.07	1.81e-01
	BCB	9.37	0.24	2.48e-02*
RQ2	TAM	6.44	0.13	9.22e-02
	TM	1.86	-0.04	6.02e-01
	SC	7.99	0.18	4.62e-02*
	BC	8.0	0.19	4.60e-02*
RQ3	# of Iterations	4.24	0.14	1.20e-01
	Time	20.13	0.63	1.60e-04*

and Time , with large effect size for CLOC, TPR , MCC , and Time and medium effect size for BCB , SC , and BC .

The test therefore indicates that, for at least one of the workflows, the production code, test code, or development process differs with respect to the attributes measured by the selected metrics. For example, Fig. 10 illustrates why the null hypothesis is rejected for CLOC but not for TLOC. The boxplot of CLOC for the *agentic* workflow is clearly separated from those of the other workflows, whereas the boxplots of TLOC overlap to varying degrees across all workflows.

5.1 RQ1. *How does the quality of the production code created with different interaction models vary?*

To answer RQ1, we separately analyse syntactic similarity (RQ1.1), functional correctness (RQ1.2) and structural complexity (RQ1.3). In the following, we denote with **A** and **B** any two sets of values, selected from the four sets of values obtained for a given metric in the four workflows.

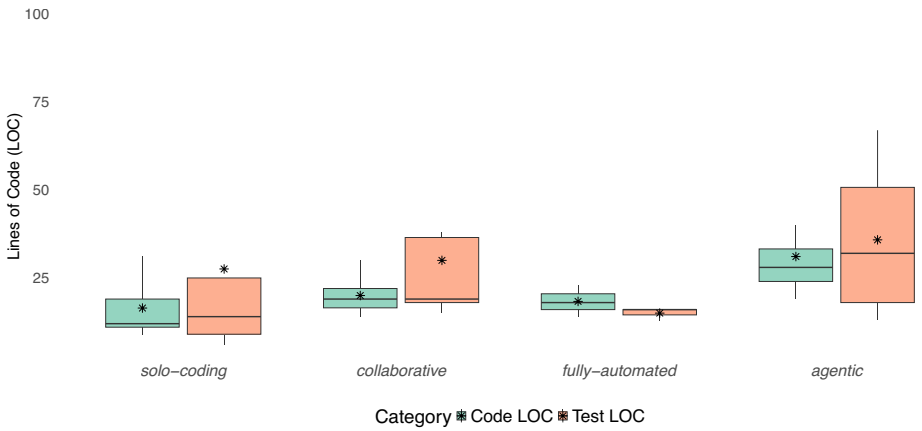


Fig. 10 Boxplots of code and TLOC grouped by workflow. The asterisk (*) denotes the mean value

5.1.1 RQ1.1. *How does the syntactic similarity of the production code created with different interaction models vary?*

In Fig. 11, we visualize the distributions of pairwise syntactic similarity of the production code measured by LCS and LED . For LCS , higher values indicate greater similarity, whereas for LED , lower values indicate greater similarity. The figure shows that the *agentic* workflow preserves longer common code fragments with the other workflows (Large LCS) while being locally distant (large LED). For example, the code generated with the *agentic* workflow preserves the main implementation but adds helper methods, logging, and validation. Locally, *collaborative* , *fully-automated* , and *solo-coding* workflows are very similar (low LED) and between *solo-coding* and either of these two workflows, but they do not share large common sequences.

Overall, the *agentic* workflow preserves longer common code fragments with the other workflows while exhibiting more localized syntactic differences. In contrast, the code produced by controlling the prompts (*collaborative* and *fully-automated* workflows) is locally closer, as well as closer to the code produced in *solo-coding* . Combined with the CLOC distributions in Fig. 10, we then summarize as in the following:

> The results indicate that the code generated by the *agentic* workflow preserves long common code fragments while introducing more local modifications. In contrast, the *solo-coding* , *collaborative* , and *fully-automated* workflows produce smaller implementations that differ mainly through localized edits.

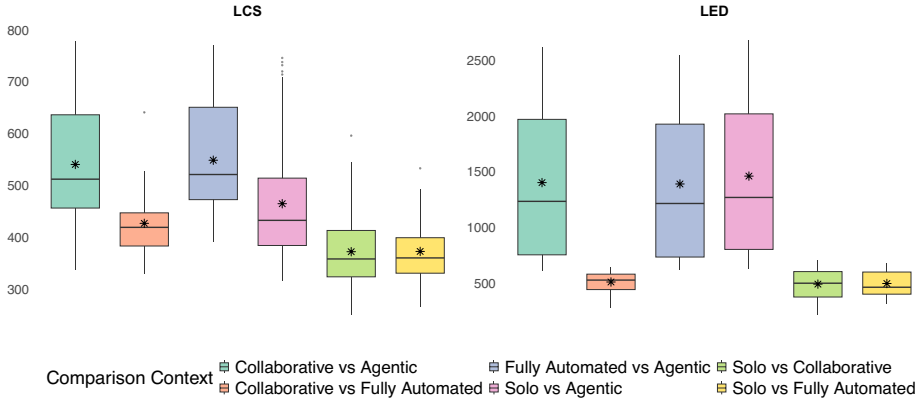


Fig. 11 Syntactic similarity between any two sets A and B for LCS and LED . LCS , the higher the better; LED , the smaller the better. (*) denotes the mean value

Table 9 One-tailed Mann–Whitney U tests for functional correctness with null hypothesis$H_0 : A > B$. (*) denote single-test statistical significance at$\alpha = 0.05$. (**) denotes statistical significance after Holm-Bonferroni correction

TPR			
$A > B$	U	Effect size	p (one-tailed)
collaborative > solo-coding	36.5	-0.16	3.14e-01
fully-automated > solo-coding	18.0	0.33	2.25e-01
collaborative > agentic	69.5	-0.65	8.55e-03**
fully-automated > agentic	36.0	-1.00	3.79e-03**
agentic > solo-coding	106.5	0.97	7.81e-05**
collaborative > fully-automated	12.0	-0.14	4.08e-01

5.1.2 RQ1.2. *How does the functional correctness of the production code created with different interaction models vary?*

The pairwise U test in Table 9 reports significant differences between the *agentic* workflow and the other workflows for the metric TPR , also after HB correction, with high effect size[9].

By manually inspecting the data in Table 16 in the appendix, we observe that the *solo-coding* and *collaborative* workflows exhibit greater variability in TPR . For example, participant P13 in the *collaborative* workflow achieved a TPR of 100.0%, whereas participant P12 achieved only 16.7% because the generated test code did not compile. Similarly, in the *solo-coding* workflow, participant P7 obtained the lowest TPR (8.3%) after creating an assertion that did not match the production logic. On average, TPR is higher with agentic coding (94.58%).

> The solutions produced by the *agentic* workflow show the highest functional correctness, although human developers can reach the same level of correctness in *collaborative* settings. Workflows involving human developers show greater variation in functional correctness.

5.1.3 RQ1.3. *How does the complexity of the production code created with different interaction models vary?*

The pairwise U test in Table 10 reports significant differences between the *agentic* workflow and the other workflows for the metric MCC , also after HB correction, with high effect size. It also shows significant differences between the *agentic* workflow and the workflows involving humans for the metric BCB , also after HB correction, with a high effect size. This indicates that the solutions produced by the *agentic* workflow contain a greater number of branches that have a lower coverage of the baseline test suite than the one developed by humans.

In Table 16, we also see that the statements and branches of the *agentic* workflow solutions are covered up to at most 75% by the baseline tests, while many participants in the *solo-coding* and *collaborative* settings achieve very high branch coverage, sometimes even reaching 100%. For example, participant P13 achieves a TPR of 100.0% with a MCC of 8, demonstrating that high functional correctness can be achieved with relatively low structural complexity in the *collaborative* workflow.

> The solutions produced under the *agentic* workflow show more complex control-flow structures than the human-developed solutions. This increased complexity may contribute to a lower branch coverage of the baseline tests than the humans developed solutions.

[9] Negative effect size flips the direction of the test, *i.e.,* from $A > B$ to $B > A$.

Table 10 One-tailed Mann-Whitney U tests for code complexity with null hypothesis$H_0 : A > B$. The asterisk (*) denote statistical significance at$\alpha = 0.05$. (**) denotes statistical significance after Holm-Bonferroni correction

MCC			
$A > B$	U	Effect size	p (one-tailed)
collaborative > solo-coding	32.5	0.03	4.79e-01
fully-automated > solo-coding	11.5	-0.15	6.85e-01
collaborative > agentic	74.5	-0.77	3.13e-03**
fully-automated > agentic	36.0	-1.00	5.48e-03**
agentic > solo-coding	102.5	0.90	2.97e-04**
collaborative > fully-automated	12.0	0.14	4.08e-01
SCB			
collaborative > solo-coding	34.0	0.08	4.15e-01
fully-automated > solo-coding	12.0	0.11	6.45e-01
collaborative > agentic	61.5	-0.46	5.37e-02
fully-automated > agentic	12.0	-0.33	8.26e-01
agentic > solo-coding	82.5	0.53	2.29e-02*
collaborative > fully-automated	11.0	0.05	5.00e-01
BCB			
collaborative > solo-coding	25.0	-0.21	7.93e-01
fully-automated > solo-coding	13.0	-0.04	5.87e-01
collaborative > agentic	71.0	0.69	7.55e-03**
fully-automated > agentic	24.0	0.33	2.11e-01
agentic > solo-coding	91.5	-0.69	3.78e-03**
collaborative > fully-automated	9.0	0.14	6.87e-01

5.2 RQ2. *How does the quality of the tests created with different interaction models vary?*

To answer RQ2, we compare the structural complexity of the test code (RQ2.1) and its code coverage with the one of the baseline suite (RQ2.2).

5.2.1 RQ2.1. *How does the structural complexity of tests created with different interaction models conform to that of the baseline tests?*

To answer RQ2.1, we analyse the structural complexity of the tests created in the workflows. The U tests in Table 11 are only significant under $\alpha = 0.05$ for TAM in two pair-wise combinations involving the *agentic* model with high effect size, but not significant after HB correction. Thus, to understand better whether the organisation of tests varies across the four groups, we qualitatively analyse the test code in the different groups, also in comparison with the organisation we derive for the baseline tests. The baseline tests are indeed organised in two test methods (one for each method to test) with five and seven assertions respectively.

The values in Table 16 show different testing styles, none of them perfectly matching that of the baseline tests. The *fully-automated* model is the poorest, as it consistently generates one assertion per test method and three test methods in total. The tests are rather incomplete. The tests generated with the *agentic* workflow are poor in the case where the prompt is more structured (Prompt 2 in Table 7). In this case, both agents (ChatGPT and Claude) have a rather constant output with one or two assertions per method and three to six methods. In the case of a less structured prompt (Prompt 1 in Table 7), the output over multiple runs of the agents is less deterministic, also providing a method with 11 assertions. A test structure closer to the baseline tests is visible in the output of the *collaborative* workflow. It is also worth noting that the baseline test suite does not include a test for the setter method (`setLineWidth`). Nevertheless, most human developers chose to test the setter, except for P10, P12, and P13, whose test suites result therefore structurally closer to the baseline suite. The *fully-automated* workflow also omits the test for the setter in all its runs. In contrast, the *agentic* workflow alternates between including and omitting setter tests across different runs, contributing to the observed variability in both TAM and TM .

Some participants (*e.g.,* P1, P10, and P11) developed comprehensive test suites covering valid, invalid, and edge cases. Others, including P2, P5, P6, and P8, focused primarily on valid and invalid inputs. We also observed attempts to improve the interaction with the CLLM through iterative test design. For example, P15 correctly identified two edge cases but, for a third one, only documented the failing scenario in the test code comments, as shown in Listing . Despite repeated interactions with the CLLM, the corresponding issue in the production code remained unresolved.

> The organization of the test suites developed in the four workflows varies. The test suites generated by the *fully-automated* and *agentic* workflows are relatively less structured, whereas participants in the *collaborative* workflow produced test suites whose organization is closer to the one of the baseline suite.

Table 11 One-tailed Mann-Whitney U tests for test structural complexity with null hypothesis$H_0 : A > B$. The asterisk (*) denote statistical significance at$\alpha = 0.05$. (**) denotes statistical significance after Holm-Bonferroni correction

$A > B$	U	Effect size	p (one-tailed)
TAM			
collaborative > solo-coding	41.0	0.30	1.54e-01
fully-automated > solo-coding	7.5	-0.44	9.24e-01
collaborative > agentic	36.5	-0.13	6.97e-01
fully-automated > agentic	31.5	-0.75	2.59e-02*
agentic > solo-coding	79.0	0.46	3.64e-02*
collaborative > fully-automated	16.5	0.57	7.89e-02
TM			
collaborative > solo-coding	38.0	-0.21	2.48e-01
fully-automated > solo-coding	10.5	-0.22	7.80e-01
collaborative > agentic	52.0	0.24	2.01e-01
fully-automated > agentic	16.5	-0.08	6.24e-01
agentic > solo-coding	46.5	-0.14	7.27e-01
collaborative > fully-automated	16.5	0.57	7.81e-02

Table 12 One-tailed Mann-Whitney U tests for code coverage with null hypothesis$H_0 : A > B$. (*) denote statistical significance at$\alpha = 0.05$. (**) denotes statistical significance after Holm-Bonferroni correction

$A > B$	U	Effect size	p (one-tailed)
SC			
collaborative > solo-coding	25.5	-0.19	7.99e-01
fully-automated > solo-coding	9.0	-0.33	8.65e-01
collaborative > agentic	61.0	0.45	5.65e-02
fully-automated > agentic	29.0	0.61	6.38e-02
agentic > solo-coding	88.5	-0.64	6.42e-03**
collaborative > fully-automated	10.5	0.0	5.49e-01
BC			
collaborative > solo-coding	26.0	-0.17	7.80e-01
fully-automated > solo-coding	8.0	-0.41	9.07e-01
collaborative > agentic	64.5	0.54	2.71e-02*
fully-automated > agentic	25.5	0.42	1.48e-01
agentic > solo-coding	87.5	-0.62	7.31e-03**
collaborative > fully-automated	12.0	-0.14	4.04e-01

5.2.2 RQ2.2. How does the Coverage of the Code Created with Different Interaction Models Vary?

To answer RQ2.2, we analyse the code coverage of the tests created in the workflows. After applying the HB correction, only two pairwise comparisons remain statistically significant (Table 12), corresponding to the SC and BC metrics. In both cases, the significant difference is between the *agentic* and *solo-coding* workflows. The test suites generated in the *agentic* workflow achieve lower statement and branch coverage on their corresponding generated production code than the test suites developed by participants in the *solo-coding* workflow on their own production code.

Listing 5 Comment that documents which assertions fails, giving more context to the CLLM.

```
# It is not working in situations where I exceed the total
number of characters allowed.
def test_words_in_the_edges(self):
    tf = TextFormatter();
    tf.setLineWidth(20);
    self.assertEqual("foo                toy", tf.
getWordsInTheEdgesOfLine("foo", "toy"))
    #self.assertEqual("pneumonoultramicrosc", tf.
getWordsInTheEdgesOfLine("pneumono", "ultramicroscopic"))
    self.assertEqual("runtime                 ", tf.
getWordsInTheEdgesOfLine("runtime", ""))
    self.assertEqual("              runtime", tf.
getWordsInTheEdgesOfLine("", "runtime"))
    self.assertEqual("                        ", tf.
getWordsInTheEdgesOfLine("", ""));
```

Human involvement remains beneficial for achieving higher code coverage. In particular, the test suites developed in the *solo-coding* workflow achieve higher statement and branch coverage than those generated in the *agentic* workflow. Moreover, enhancing the prompt used to trigger the activities of the *agentic* workflow does not necessarily improve code coverage.

5.3 RQ3. How does the Efficiency of the TDD Process Vary Between Different Interaction Models?

To answer RQ3, we analyse two process metrics: # of Iterations and Time . For the *agentic* workflow iterations were not collected. The remaining pairwise U tests are illustrated in Table 13. The table shows that Time is significantly lower in the *agentic* workflow than in any other workflow, with a large effect size in all comparisons, even after the HB correction. Furthermore, Time in the *fully-automated* workflow is significantly lower than in the *collaborative* workflow.

Table 14 provides additional insights into how the collaborative iterations unfold. Besides the total number of iterations, the table reports the number of successful iterations and the number of iterations requiring assertion rework, *i.e.,* when the toolset reached the maximum limit of five repetitions and returned control to the user to clarify the intended behaviour to the CLLM. On average, the *collaborative* workflow required 12.1 iterations, of which

Table 13 One-tailed Mann-Whitney U tests for process efficiency with null hypothesis$H_0 : A > B$. The asterisk (*) denote statistical significance at$\alpha = 0.05$. (**) denotes statistical significance after Holm-Bonferroni correction

$A > B$	U	Effect size	p (one-tailed)
# of Iterations			
collaborative > solo-coding	49.0	-0.56	3.57e-02*
fully-automated > solo-coding	6.0	-0.56	9.31e-01
collaborative > agentic	-	-	-
fully-automated > agentic	-	-	-
agentic > solo-coding	-	-	-
collaborative > fully-automated	8.5	0.19	7.16e-01
Time			
collaborative > solo-coding	30.5	-0.03	5.65e-01
fully-automated > solo-coding	24.0	-0.78	3.15e-02*
collaborative > agentic	84.0	1.00	9.37e-05**
fully-automated > agentic	36.0	1.00	2.08e-03**
agentic > solo-coding	94.5	-0.75	1.28e-03**
collaborative > fully-automated	21.0	1.00	8.97e-03**

Table 14 Logs of the iterations on the *collaborative* model

ID	# of Iterations	successful	with reworked assertions
P10	10	5	6
P11	11	1	4
P12	12	2	3
P13	26	9	12
P14	8	2	4
P15	14	4	6
P16	4	4	4
Mean	12.1	3.9	5.6

Table 15 Grouped post-experiment responses for exercise perception by interaction model

	Easy	Fine	Hard
solo-coding	P5, P6, P7, P9	P1, P3, P4	P2, P8
collaborative	P10, P11, P12, P13	P14, P15, P16	–

3.9 were successful and 5.6 involved assertion rework. These results indicate that a lower number of iterations does not necessarily correspond to a smoother development process, as participants frequently had to reformulate or refine their tests before the CLLM could proceed correctly. The data obtained from participants P13 and P16 illustrate this variability well. P13 completed 26 iterations, including 12 iterations for assertion rework, whereas P16 completed only four iterations, all of them required rework.

> The *agentic* workflow achieves the shortest development time, significantly outperforming all other workflows, while the *fully-automated* workflow is also significantly faster than the *collaborative* workflow. However, the analysis of collaborative iterations in the *collaborative* workflow shows that development effort is not solely reflected by the number ofiterations, as participants frequently had to reformulate assertions before the CLLM could proceed, leading to substantial variability across users.

5.4 Feedback

The post-experiment questionnaire collected: (1) how the participants perceived the task and (2) a free text field in which the participant could freely express their feedback.

Table 15 illustrates, most participants in both scenarios rated the exercise as easy or fine with the knowledge they had, while only P2 and P8 in the *solo-coding* setting reported it as hard. At the same time, this perception does not fully align with the quantitative results: several participants who underperformed still rated the exercise as easy or fine, such as P4, P7, P9, P12, P14, and P16.

Participants P5, P13, P14, and P16 reported a positive experience with the experiment. P5 described it as a “Fun coding challenge”, P13 stated that “It was fun and seems like a great idea”, and P14 commented “I like the experiment, the tool seems easy to use”. P16 also expressed appreciation for the interaction with the tool, noting that “The tool works,

I enjoyed interacting with it", while still observing that "The colab platform wasn't that smooth". In general, the positive feedback mainly concerns the engaging nature of the task and the interaction with the CLLM in the *collaborative* mode. However, participant P9 noted that "1. I was too focused on writing good comprehensive code and did not look at the time. 2. I spent too much time on the setLineWidth function 3. The auto-complete in the Browser was much slower than expected. ". They were unable to complete the task within the given time frame, as they focused on producing high-quality code and consequently lost track of time. Additionally, P9 and P16 highlighted difficulties in interacting with the environment, namely, Google Colab, indicating that future studies should consider using more seamlessly integrated platforms. Furthermore, P10 stated the following "To be honest, the "presence" of the AI made me a little unsure in the beginning, because I was concerned about its behaviour and if I should adapt to fit its need. Once I realized the AI would adapt to my needs (in particular my dev-flow), I think the experience went way more smoothly." highlighting that a mental model shift needs to take place to effectively collaborate with the CLLM.

In contrast to this positive feedback, participants P8, P6, and P4 report a negative experience. P8 reported "I didn't understand the questions but after Moritz's help it was fine", P6 also stated "It took me WAY LONGER to understand what the english of the exercise meant than actually do it", and P4 expressed "maybe providing a couple of examples strings for the tests may have speed up a bit the work and clarified some requirements". When we analyse the correctness of their results against the baseline tests and inspect their code, we see that only the code of P4 underperformed with respect to the other participants and, in fact, misunderstood the task. The other two participants scored low on correctness, P7 and P9. P9 did not consider edge cases, while P7 misunderstood part of the task.

The results of the survey can be obtained in the folder `responses` within the replication package (Mock and Russo 2025).

> Most participants enjoyed the experiment, particularly the collaborative interaction with the CLLM, although some initially struggled with the open-ended task and others faced time constraints or platform difficulties, highlighting both positive participation and areas for improvement in task design and environment integration.

5.5 Summary of the Results

RQ1. *How does the quality of the production code created with different interaction models vary?* The interaction model influences the quality of the generated production code. The *agentic* workflow consistently produces functionally correct solutions that share long common code fragments but differ by local changes from the code produced in the other workflows, also at the cost of greater structural complexity and lower branch coverage by the baseline test suite. In contrast, the *solo-coding* , *collaborative* , and *fully-automated* workflows generate smaller and locally more similar implementations. Human involvement in *collaborative* models introduces greater variability in functional correctness sometimes achieving the same correctness of code generated in the *agentic* workflow.

RQ2. *How does the quality of the tests created with different interaction models vary?* The interaction model also affects the quality of the generated test suites. Human involvement leads to better-organized test suites that more closely resemble the baseline tests and, in the *solo-coding* workflow, achieves higher statement and branch coverage than the *agentic* workflow. In contrast, the *fully-automated* and *agentic* workflows produce less structured test suites, indicating that increasing prompt structure alone does not necessarily improve test quality or code coverage.

RQ3. *How does the efficiency of the TDD process vary between different interaction models?* The *agentic* workflow is the most time-efficient, followed by the *fully-automated* workflow. In contrast, the *collaborative* workflow requires frequent assertion reformulation during the interaction with the CLLM, indicating additional coordination effort despite the benefits of human involvement.

6 Discussion

Our evaluation combines two complementary settings. The human-centred interaction models (*solo-coding* and *collaborative*) were evaluated through a controlled pre-experimental study involving professional developers, whereas the *fully-automated* and *agentic* workflows were evaluated through repeated executions of autonomous workflows on the same development tasks. Throughout the discussion, results are interpreted within the context of these respective evaluation settings.

The *agentic* workflow consistently produces the most functionally correct production code, as measured by the baseline test suite, and is also the fastest approach. However, these advantages come with greater structural complexity and lower branch coverage when evaluated using the same test suite. One possible explanation is that some of the uncovered branches correspond to implementation decisions that are not explicitly defined by the functional specifications. This may indicate that the agents in the *agentic* workflow make independent implementation decisions, introducing additional untested decision points beyond what is strictly required by the specified functionality. This suggests that autonomous *agentic* platforms can effectively solve development tasks while generating solutions that are architecturally more complex and potentially more difficult to maintain.

Human involvement remains valuable, particularly for testing. Developers may produce better-organized test suites with greater code coverage. These findings results are compared with the repeated executions of the autonomous workflows, the exploratory evidence suggests that human judgment is especially beneficial for designing effective tests, while simply increasing prompt sophistication or delegating the process to autonomous agents does not automatically improve testing quality. The collaborative workflow represents a compromise between automation and human expertise. It can achieve a production code correctness comparable to the *agentic* workflow while producing higher-quality test suites. However, this benefit comes at the expense of efficiency, as developers frequently needed to reformulate assertions and coordinate with the CLLM, increasing the overall development effort.

The *fully-automated* workflow offers shorter development times than human-assisted approaches but does not achieve the same level of production code correctness as the agen-

tic workflow nor the same testing quality as workflows involving humans. This suggests that prompt-driven automation alone is insufficient to match the reasoning capabilities of autonomous agentic systems or the testing skills provided by developers (Table 16).

The participants' feedback complements these quantitative findings. Most developers reported a positive experience, particularly appreciating the collaborative interaction with the CLLM, indicating that such workflows are acceptable and engaging despite requiring additional effort. At the same time, reports of difficulties with the open-ended task, time pressure, and platform usability point to practical limitations that may have influenced efficiency and highlight opportunities to improve experimental environments and development tools.

7 Threats to Validity

The major threats we envisage in our work are related to construct, internal, and external validity.

Construct Validity Different evaluation settings. This study combines two complementary empirical settings. The *solo-coding* and *collaborative* workflows were evaluated through a controlled pre-experimental study involving professional developers, while the *fully-automated* and *agentic* workflows were evaluated through repeated executions of the tasks. This distinction reflects the intrinsic characteristics of the evaluated interaction models: autonomous workflows do not involve human decision making during execution and therefore cannot be assessed through the same experimental protocol adopted for human-centred workflows. To facilitate comparison, all workflows were evaluated using the same development tasks, quality metrics, and evaluation framework. Nevertheless, comparisons between human-centred and autonomous workflows should be interpreted in light of their different evaluation settings.

Internal Validity Threats in this category are related to confounding factors that affect the study's findings. In our work, the results may have been affected by the experience in TDD and Python of the developers. TDD developers must have a strong command of the practice and experience in development, Causevic et al. (2011). To mitigate this issue, we selected professionals with specific experience in TDD and designed two simple tasks for which basic programming knowledge was needed.

The sample size of the experiment with humans may be considered limited. Sampling in software engineering research studies is, in fact, an inherently difficult problem (Baltes and Ralph 2022) as well as recruiting professionals to participate in TDD experiments (Ghafari et al. 2020; Romano et al. 2025). Thus, we made sure that we achieved the typical sample size of studies with professionals in software engineering, Kitchenham and Madeyski (2024). Nonetheless, the limited number of professional participants reduces the statistical power of our analyses. As increasing the sample size was not feasible, we adopted non-parametric statistical tests (Kruskal–Wallis and Mann–Whitney), which are appropriate for small samples and do not rely on normality assumptions. We also report effect sizes and

apply multiple-testing correction to provide readers with a broader range of evidence on which to interpret the results.

The instrumentation, baseline test suite, model configuration, TDD process, and experimental environment may all have influenced the quality of the collected data and, consequently, the study results. As this work is exploratory, we acknowledge this as a relevant threat, although addressing it exhaustively was beyond the scope of the study. To facilitate future replication, we described our methodology in a way that is as independent as possible from these implementation-specific factors. Investigating the impact of variations in these factors is left for future work.

External Validity External validity concerns the ability to generalise the results. The development tasks we gave in our experiments are simple, but they are suitable in this exploratory study as they gave us the possibility to vertically inspect the output to interpret the quantitative tests. In addition, the tasks were designed carefully to allow participants with different skills and experience to produce the code in a short time (40 min).

We used a single CLLM (ChatGPT) accessed through API calls. This choice ensured a consistent experimental setting and enabled a more reliable comparison of the outputs across participants. To improve the generalisability of our findings, we also evaluated the agentic workflow using two different CLLMs, allowing us to assess whether its performance was consistent across models.

8 Related Work

There is an ongoing debate about the role of GenAI in software development. In this overview, we summarise recent research along three main directions: GenAI as a tool for code generation, as a peer collaborator, and as an enabler of the TDD process.

GenAI as a Tool GenAI has the potential to significantly enhance software development productivity (Fan et al. 2023). For example, pair programming with AI assistants can increase development speed by up to 55.8% (Peng et al. 2023). Pre-trained LLMs are frequently fine-tuned for specific generation tasks, such as producing secure code (Li et al. 2024), generating unit tests (Tufano et al. 2022), or summarising code (Mastropaolo et al. 2024). Putting attention to parameter optimisation increases the performance of fine-tuning for code generation compared to other approaches (Weyssow et al. 2025) and is particularly effective when the training data is limited (Giagnorio et al. 2025). At the same time, direct prompting pre-trained LLMs for code generation is gaining momentum (Brown et al. 2020; Wang et al. 2024). This approach is less resource-intensive than fine-tuning but may suffer from limited domain knowledge. As it does not require much background knowledge, inexperienced users may produce low-quality code without knowing it. Determining which approach (fine-tuning or prompting) is superior remains an open question, and keeping humans in the loop is generally considered advisable (Shin et al. 2025; Mock et al. 2025). Regardless of the approach, the quality of the generated code is a key concern (Mailach et al. 2024; Shin et al. 2025). One strategy is to prompt LLMs iteratively and repeatedly to refactor and improve

the code (Fakhoury et al. 2024; White et al. 2024). This solution requires considerable effort and may not be feasible for large, complex development projects. Our *collaborative* and *fully-automated* approaches aim to address this problem by automating collaboration and prompting.

GenAI as a Peer Collaborator Vibe coding and agentic coding provide different perspectives of human-GenAI collaboration in software development, Sapkota et al. (2025). Much of the current research on code generation focuses on vibe coding, where GenAI acts as a full-fledged assistant with whom developers can engage in a real conversation. indeed, CLLMs have been introduced into software development to support this goal (Peng et al. 2023). Without a thorough understanding of the code, conversations with CLLMs can again lead to low-quality code. Although vibe coding is fast and accessible, it comes with the costs of low quality (*e.g.,* technical debt), little attention to quality assurance (*e.g.,* no testing), and poor prompt design (Fawzy et al. 2025). This situation resembles the early stages of open source development, and we believe that the strategies developed since then (Champion and Hill 2025) can also be effectively applied in this context.

Since 2020, agentic coding has emerged as a new frontier in software development (Sapkota et al. 2025). Agentic coding refers to the paradigm in which LLMs act as autonomous agents capable of generating code with minimal human intervention. Building on this concept, agentic platforms coordinate multiple pre-trained LLMs (agents) that collaborate to achieve predefined software development goals. These platforms enable the selection and orchestration of different models to generate diverse code artifacts, effectively simulating an end-to-end software engineering workflow. The LLM agents (*e.g.,* AutoGPT Richards, 2023, BabyAGI Nakajima, 2023, Devin AI Labs, 2024, OpenAI Codex Openai, 2021, Google Jules Deepmind, 2025) demonstrate the ability to decompose abstract goals, generate code, invoke APIs, interact with development environments, and reason iteratively through planning-execution-feedback loops. MetaGPT (Hong et al. 2024) is a multi-agent platform that allows users to select the type of LLM as an agent and provides a fully automated platform for development, where the user just needs to express a development goal, and the agents act as developers working individually or in a team. All these initiatives are built on predefined development processes. For instance, MetaGPT X relies on development workflows (Standard Operating Procedure - SOPs) that are unable to capture the iterative and incremental nature of Test-Driven Development. In our work, we introduced models for process development that not only automate tasks but also structure and manage interactions with CLLMs within iterative workflows. Based on our preliminary findings, we argue that the software development process itself requires rethinking. The various phases of the software life cycle may need to be redesigned to fully leverage the rapid development capabilities of CLLMs. For example, atomic design elements and functional requirements could be directly incorporated into the prompts, while quality assurance mechanisms should be redefined to automatically control the generated artefacts during the development process.

GenAI and TDD To the best of our knowledge, three approaches are relevant for our work: LLM4TDD that develops into two subsequent studies (Piya and Sullivan 2024; Piya et al. 2025), TGen (Mathews and Nagappan 2024), and TiCoder (Fakhoury et al. 2024). For all of them, the programming language is Python.

Our approach focuses on integrating the collaboration with CLLMs into the TDD process and empirically assesses the effects on software quality. The goals of LLM4TDD and TGen somehow target the opposite problem by studying how the information coming from the tests and their execution through a TDD process can improve the ability of LLM to generate correct code. The goal of TiCoder is to help users select the most correct code among a list of LLM-generated snippets. To achieve our goal, we conducted two studies: (i) an experiment involving 16 professionals experienced in TDD, and (ii) a comparative analysis of the correctness of the code generated automatically during the experiment and the code generated using an agentic development platform.

Instead, LLM4TDD and TGen perform case studies mining public datasets on which their framework is applied for code generation and analysis. The authors of TiCoder designed an interactive TDD workflow in which users prompt the LLM for a task, and the model generates a set of candidate code and test suggestions. User feedback is then incorporated to refine the suggestions for code whose tests fail. In TiCoder, the experiment was run with a similar number of participants, time frame for the experiment run and with an additional pre-experiment trial to ours. However, it was conducted with nine PhD students and six professionals from Microsoft and the pre-experiment trial involved three professionals. We follow a low-granularity version of TDD, in which each TDD iteration executes one assertion at a time, *i.e.,* assertion-first. Both LLM4TDD and TGen perform TDD iterations based on failing tests, which may include multiple assertions per test case. In TiCoder, LLM generates tests and code based on initial tasks. The iteration is performed to incorporate user's feedback. To evaluate the accuracy of the respective approach, our work, LLM4TDD and TGen define a set of tests as ground truth. In LLM4TDD, tests are developed manually using partition, boundary, and state-based testing and generated with LLM. TGen, on the other hand, relies entirely on existing unit tests developed by humans from MBPP and HumanEval, plus synthetically generated ones from EvalPlus. In TiCoder, tests are generated by the LLM and refined with the user's feedback. Also in this case, the iteration is triggered when some tests fail. In addition, TiCoder uses the MBPP and HumanEval tests to evaluate the accuracy of the generated code. Our baseline tests are the ground truth for all our experiments. They are crafted manually through category partition testing and are derived from specifications.

LLM4TDD counts the number of prompts against the number of ground truth tests and correctness. TGen focuses primarily on correctness using the percentage of passed tests in different research problem settings. TiCoder uses time and cognitive load to measure the performance of developers in evaluating AI output and pass@k of the ground truth to measure the accuracy of the generated code. LLM4TDD and TGen find that more than 80% of the problems can be solved without reiterating with prompts and hinting with human intervention or more information on the test and their execution. TiCoder finds that incorporating users' intent boosts LLM performance to generate correct code. These results are

based on existing datasets of problems and a limited number of tests (up to 4 for MBBP and HumanEval with TGen), tests and iteration (for LLM4TDD, up to 10 cases and iterations) or tasks and related tests (for TiCoder). Our work does not set limits on iterations or in the process of generating the baseline tests. Our qualitative analysis also suggests that intensive collaboration with a CLLM can produce functionally correct code, performing as well as an *agentic* platform, although the *agentic* workflow statistically produces higher-quality code.

9 Conclusions and Future Work

This study presents an exploratory investigation of human–AI collaboration in software development. We implemented and evaluated different interaction models, assessing their impact on the quality of the resulting software artifacts. We conducted a controlled pre-experimental study involving professional developers to evaluate the *solo-coding* and *collaborative* workflows. Then, we replicated the study with *fully-automated* and *agentic* workflows to provide exploratory evidence on autonomous software generation. To compare the output resulting from the four models of interaction, we developed an evaluation framework to assess quality of code and process efficiency.

Our findings indicate that the choice of an interaction model should depend on the development objective. Agentic workflows are particularly suitable when rapid development and production code correctness are the primary goals. However, they may also introduce additional implementation decisions that are not explicitly required by the functional specifications, resulting in untested decision points. Conversely, workflows involving humans remain preferable when high-quality testing, better-organized test code, and greater control over the development process are required. Collaborative approaches offer a promising balance between these objectives but require improved interaction mechanisms to reduce the coordination overhead between developers and CLLMs.

In future work, we plan to extend our investigation to additional development practices, CLLM, and agentic platforms, while also exploring how AI may reshape the software development process itself. We aim to investigate whether traditional phase-based and iterative development models remain appropriate in an era where AI systems can generate complete software solutions without explicitly progressing through conventional phases such as design.

We also plan to apply our *fully-automated* evaluation suite to established benchmarks such as MBPP (Austin et al. 2021) and HumanEval (Chen et al. 2021), which provide problem specifications together with reference implementations and test suites, to further assess the quality of the generated software artifacts.

Appendix

Table 16 Metrics' values per participant (vibe coding) and per run (agentic coding) and means and standard deviations. The brackets for the values of TAM indicate the total number of assertions. (*) Participants perceiving time >40 min (actual limit: 40 min). Abbreviations: CG (ChatGPT), CS (Claude Sonnet), FA (*fully-automated*)

	Production code				Test code				Process	
ID	TPR (%)	MCC	SCB (%)	BCB (%)	TAM (Ass.)	TM	SC (%)	BC (%)	Iter.	Time (sec)
solo-coding										
P1	83.3	9	65	88	1.55 (17)	11	100	100	26	2,100
P2	83.3	5	64	100	1 (3)	3	100	100	13	2,400
P3	75.0	6	64	100	1.5 (6)	4	91	75	22	1,200
P4	33.3	7	53	100	3 (3)	1	67	25	25	2,100
P5	83.3	4	64	100	1 (3)	3	100	100	12	2,400
P6*	75.0	8	14	0	1 (3)	3	100	100	22	4,200
P7	8.3	7	45	100	1 (3)	3	100	100	9	600
P8*	75.0	5	54	100	1 (3)	3	100	100	21	3,600
P9	41.7	7	53	67	1.27 (14)	11	100	100	29	1,800
Mean	62.0	6.44	52.89	83.89	1.37 (6.11)	4.67	95.33	88.89	19.89	2,266
SD	27.3	1.59	16.17	33.35	0.65 (5.47)	3.67	11.03	25.34	6.94	1,103
collaborative										
P10	91.7	9	64	83	1 (7)	7	91	83	10	1,800
P11	83.3	4	42	100	3 (9)	3	100	100	11	1,800
P12	16.7	4	67	100	1 (4)	4	100	100	12	2,400
P13	**100.0**	**8**	**62**	**83**	**6 (18)**	**3**	**100**	**100**	**26**	**2,400**
P14	33.3	6	64	75	1.6 (8)	5	86	50	8	2,400
P15	83.3	6	56	100	2.67 (8)	3	100	100	14	2,400
P16	16.7	9	31	12	1 (5)	5	85	62	4	1,200
Mean	60.8	6.57	55.14	79.00	2.32 (8.43)	4.29	94.57	85.00	12.14	2,057
SD	36.8	2.15	13.55	31.25	1.82 (4.58)	1.50	7.02	21.05	6.89	472

Table 16 (continued)

	Production code				Test code				Process	
fully-automated										
FA1	83.3	4	36	100	1 (3)	3	100	100	4	720
FA2	83.3	7	50	33	1 (3)	3	94	67	18	960
FA3	83.3	7	77	100	1 (3)	3	91	75	16	720
Mean	83.3	6.0	54.33	77.67	1 (3)	3	95.00	80.67	12.67	800
SD	0.0	1.73	20.84	38.68	0 (0)	0	4.58	17.21	7.57	138
agentic										
CG1_1	83.0	17	68	62	1.44 (13)	9	100	100	-	120
CG1_2	100.0	15	64	62	11 (11)	1	97	94	-	120
CG1_3	100.0	13	58	58	10 (10)	1	94	83	-	120
CG2_1	92.0	12	60	60	1 (3)	3	80	50	-	60
CG2_2	92.0	9	62	67	1 (3)	3	88	50	-	60
CG2_3	92.0	8	61	75	1 (3)	3	89	50	-	60
CS1_1	100.0	13	70	71	1.5 (15)	10	93	86	-	60
CS1_2	100.0	16	52	60	1.8 (18)	10	71	80	-	60
CS1_3	92.0	12	71	70	14 (14)	1	86	60	-	60
CS2_1	100.0	9	75	75	2 (3)	6	86	50	-	60
CS2_2	92.0	8	69	62	2 (8)	4	85	50	-	60
CS2_3	92.0	8	69	62	2 (6)	3	86	50	-	60
Mean	94.58	11.67	64.92	65.33	4.06 (9.17)	4.25	87.92	66.92	-	75.0
SD	5.40	3.26	6.54	6.02	4.69 (5.13)	3.41	7.75	19.97	-	27.14

Acknowledgements The research has been funded by the European Commission Next Generation EU as part of the project PRIN2022 2022TEPX4R - Behaviour-enabled IoT (BeT), CUP 53D23003730008 and the European Union- Next Generation EU, Mission 4 Component 1 CUP I52B23000570003.

Funding Open access funding provided by Libera Università di Bolzano within the CRUI-CARE Agreement. The research has been funded by the European Commission Next Generation EU as part of the project PRIN2022 2022TEPX4R - Behaviour-enabled IoT (BeT), CUP 53D23003730008 and the European Union-Next Generation EU, Mission 4 Component 1 CUP I52B23000570003.

Data Availability The toolbox, alongside the collected data from participants during the experiment, along with the feedback obtained through the questionnaire which was conducted after the experiment, are available in the GitHub replication package (Mock and Russo, 2025).

Declarations

Conflict of Interest The authors declare no conflict of interest.

Ethical approval This study did not require formal approval from an ethics committee. The research was conducted in accordance with the principles of good scientific practice.

Informed consent All participants were informed about the purpose of the study and provided their consent prior to participation through the online questionnaire.

Clinical trial Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Astromskis S, Bavota G, Janes A, Russo B, Di Penta M (2017) Patterns of developers behaviour: A 1000-hour industrial study. J Syst Softw 132:85–97. https://doi.org/10.1016/j.jss.2017.06.072, https://www.sciencedirect.com/science/article/pii/S016412121730136X

Austin J, Odena A, Nye M, Bosma M, Michalewski H, Dohan D, Jiang E, Cai C, Terry M, Le Q, Sutton C (2021) Program synthesis with large language models. https://doi.org/10.48550/arXiv.2108.07732

Baltes S, Ralph P (2022) Sampling in software engineering research: a critical review and guidelines. Empir Softw Eng 27(4):94. https://doi.org/10.1007/s10664-021-10072-8

Beck K (2022) Test driven development: By example. Addison-Wesley

Bishop CM, Bishop H (2023) Deep learning: foundations and concepts. Springer Nature. https://doi.org/10.1007/978-3-031-45468-4

Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler DM, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D (2020) Language models are few-shot learners. In: Advances in neural information processing systems, Vancouver, BC, Canada. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

Butler J, Suh J, Haniyur S, Hadley C (2025) Dear diary: A randomized controlled trial of generative ai coding tools in the workplace. In: 2025 IEEE/ACM 47th international conference on software engineering: software engineering in practice (ICSE-SEIP), Ottawa, ON, Canada, pp 319–329. https://doi.org/10.1109/ICSE-SEIP66354.2025.00034

Caldiera V, Rombach HD (1994) The goal question metric approach. Encyclopedia of Software Engineering

Campbell DT, Stanley JC (2015) Experimental and quasi-experimental designs for research.(Ravenio books)

Causevic A, Sundmark D, Punnekkat S (2011) Factors limiting industrial adoption of test driven development: a systematic review. In: Fourth IEEE international conference on software testing, verification and validation, IEEE, Berlin, BE, Germany, pp 337–346. https://doi.org/10.1109/ICST.2011.19

Champion K, Hill B (2025) Underproduction analysis of open source software. Empirical Software Engineering 30. https://doi.org/10.1007/s10664-025-10703-4

Chen M, Tworek J, Jun H, Yuan Q, de Oliveira Pinto HP, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G, Ray A, Puri R, Krueger G, Petrov M, Khlaaf H, Sastry G, Mishkin P, Chan B, Gray S, Ryder N, Pavlov M, Power A, Kaiser L, Bavarian M, Winter C, Tillet P, Such FP, Cummings D, Plappert M, Chantzis F, Barnes E, Herbert-Voss A, Guss WH, Nichol A, Paino A, Tezak N, Tang J, Babuschkin I, Balaji S, Jain S, Saunders W, Hesse C, Carr AN, Leike J, Achiam J, Misra V, Morikawa E, Radford A, Knight M, Brundage M, Murati M, Mayer K, Welinder P, McGrew B, Amodei D, McCandlish S, Sutskever I, Zaremba W (2021) Evaluating large language models trained on code. https://doi.org/10.48550/arXiv.2107.03374,

Chiang WL, Zheng L, Sheng Y, Angelopoulos AN, Li T, Li D, Zhu B, Zhang H, Jordan MI, Gonzalez JE, Stoica I (2024) Chatbot arena: an open platform for evaluating llms by human preference. In: Proceedings of the 41st International Conference on Machine Learning, JMLR.org, Vienna, Austria, ICML'24. https://dl.acm.org/doi/abs/10.5555/3692070.3692401

DeepMind G (2025) Google jules: Ai software engineering assistant. https://jules.google, last accessed: October 2025

Devlin J, Chang MW, Lee K, Toutanova K (2019) BERT: pre-training of deep bidirectional transformers for language understanding. In: NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference, vol 1, pp 4171–4186

Ebert C, Louridas P (2023) Generative AI for software practitioners. IEEE Softw 40(4):30–38. https://doi.org/10.1109/MS.2023.3265877

ERCEA (2023) Foresight: Use and impact of artificial intelligence in the scientific process. https://erc.europa.eu/sites/default/files/2023-12/AI_in_science.pdf, last accessed: May 2025

Fakhoury S, Naik A, Sakkas G, Chakraborty S, Lahiri SK (2024) Llm-based test-driven interactive code generation: User study and empirical evaluation. IEEE Trans Softw Eng 50(9):2254–2268. https://doi.org/10.1109/TSE.2024.3428972

Fan A, Gokkaya B, Harman M, Lyubarskiy M, Sengupta S, Yoo S, Zhang JM (2023) Large language models for software engineering: survey and open problems. In: 2023 IEEE/ACM international conference on software engineering: future of software engineering (ICSE-FoSE), IEEE, Melbourne, Australia, pp 31–53. https://doi.org/10.1109/ICSE-FoSE59343.2023.00008

Fawzy A, Tahir A, Blincoe K (2025) Vibe coding in practice: Motivations, challenges, and a future outlook - a grey literature review. https://doi.org/10.48550/arXiv.2510.00328

Fucci D, Erdogmus H, Turhan B, Oivo M, Juristo N (2017) A dissection of the test-driven development process: does it really matter to test-first or to test-last? IEEE Trans Softw Eng 43(7):597–614. https://doi.org/10.1109/TSE.2016.2616877

Ghafari M, Gross T, Fucci D, Felderer M (2020) Why research on test-driven development is inconclusive? In: Proceedings of the 14th ACM / IEEE international symposium on empirical software engineering and measurement (ESEM), Proceedings of the 14th ACM / IEEE international symposium on empirical software engineering and measurement (ESEM), ACM, Bari, Italy, pp 1–10. https://doi.org/10.1145/3382494.3410687

Giagnorio A, Martin-Lopez A, Bavota G (2025) Enhancing code generation for low-resource languages: no silver bullet. In: 2025 IEEE/ACM 33rd international conference on program comprehension (ICPC), Ottawa, ON, Canada, pp 478–488. https://doi.org/10.1109/ICPC66645.2025.00058

Goodfellow I, Bengio Y, Courville A (2016) Deep Learning. MIT Press. https://www.deeplearningbook.org, last accessed: October 2025

Holm S (1979) A simple sequentially rejective multiple test procedure. Scand J Stat 6(2):65–70. http://www.jstor.org/stable/4615733

Hong S, Zhuge M, Chen J, Zheng X, Cheng Y, Zhang C, Wang J, Wang Z, Yau SKS, Lin Z et al (2024) Metagpt: Meta programming for a multi-agent collaborative framework. In: International conference on learning representations, ICLR

ISO25010 (2024) 25010:2024. https://www.iso.org/standard/78175.html

Kitchenham B, Madeyski L (2024) Recommendations for analysing and meta-analysing small sample size software engineering experiments. Empirical Softw Engg 29(6). https://doi.org/10.1007/s10664-024-10504-1

Kitchenham B, Madeyski L, Budgen D et al (2017) Robust statistical methods for empirical software engineering. Empir Softw Eng 22:579–630. https://doi.org/10.1007/s10664-016-9437-5

Kruskal WH, Wallis WA (1952) Use of ranks in one-criterion variance analysis. J Am Stat Assoc 47(260):583–621. https://doi.org/10.1080/01621459.1952.10483441

Labs C (2024) Devin: the first ai software engineer. https://www.cognition.ai/blog/introducing-devin, last accessed: October 2025

Levenshtein VI (1966) Binary codes capable of correcting deletions, insertions, and reversals. Soviet Phys doklady Soviet Union 10:707–710

Li G, Hammoud H, Itani H, Khizbullin D, Ghanem B (2023) Camel: communicative agents for“ mind” exploration of large language model society. Adv Neural Inf Process Syst 36:51991–52008

Li J, Sangalay A, Cheng C, Tian Y, Yang J (2024) Fine tuning large language model for secure code generation. In: Proceedings of the 2024 IEEE/ACM first international conference on AI foundation models and software engineering, Association for Computing Machinery, Lisbon, Portugal, FORGE ’24, pp 86–90. https://doi.org/10.1145/3650105.3652299

Liu P, Yuan W, Fu J, Jiang Z, Hayashi H, Neubig G (2023) Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput Surv 55(9). https://doi.org/10.1145/3560815

Liu Z, Tang Y, Luo X, Zhou Y, Zhang LF (2024) No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. IEEE Trans Softw Eng 50(6):1548–1584. https://doi.org/10.1109/TSE.2024.3392499

Mailach A, Gorgosch D, Siegmund N, Siegmund J (2024) “ok pal, we have to code that now”: interaction patterns of programming beginners with a conversational chatbot. Empirical Softw Engg 30(1). https://doi.org/10.1007/s10664-024-10561-6

Mangiafico SS (2025) Summary and analysis of extension program evaluation in r. https://rcompanion.org/handbook, version 1.23.0, revised 2025. Accessed 20 May 2025

Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. Ann Math Stat 18(1):50–60. http://www.jstor.org/stable/2236101

Mastropaolo A, Ciniselli M, Pascarella L, Tufano R, Aghajani E, Bavota G (2024) Towards Summarizing Code Snippets Using Pre-Trained Transformers. In: Proceedings of the 32nd IEEE/ACM international conference on program comprehension, Lisbon, Portugal, pp 1–12. https://doi.org/10.1145/3643916.3644400

Mathews NS, Nagappan M (2024) Test-driven development and llm-based code generation. In: Proceedings of the 39th IEEE/ACM international conference on automated software engineering, Association for Computing Machinery, New York, NY, USA, ASE ’24, pp 1583–1594. https://doi.org/10.1145/3691620.3695527

Maynez J, Narayan S, Bohnet B, McDonald RT (2020) On faithfulness and factuality in abstractive summarization. ArXiv:2005.00661, https://api.semanticscholar.org/CorpusID:218487034

McCabe T (1976) A complexity measure. IEEE Trans Softw Eng SE–2(4):308–320. https://doi.org/10.1109/TSE.1976.233837

Meneely A (2016) Actionable metrics are better metrics. In: Menzies T, Williams L, Zimmermann T (eds) Perspectives on data science for software engineering, Morgan Kaufmann, Boston, pp 283–287. https://doi.org/10.1016/B978-0-12-804206-9.00051-9

Min B, Ross H, Sulem E, Veyseh APB, Nguyen TH, Sainz O, Agirre E, Heintz I, Roth D (2024) Recent advances in natural language processing via large pre-trained language models: a survey. ACM Comput Surv 56(2):1–40. https://doi.org/10.1145/3605943

Mock M, Russo B (2025) Vibe coding: an experiment with test-driven development – replication package. https://github.com/moritzmock/AI4TDD_journal, last accessed: October 2025

Mock M, Melegati J, Russo B (2025) Generative ai for test driven development: Preliminary results. In: Marchesi L, Goldman A, Lunesu MI, Przybyłek A, Aguiar A, Morgan L, Wang X, Pinna A (eds) Agile processes in software engineering and extreme programming - workshops. Springer Nature Switzerland, Bozen, Italy, pp 24–32

Nakajima Y (2023) Babyagi: an autonomous task management ai. https://github.com/yoheinakajima/babyagi, last accessed: October 2025

Nascimento N, Guimaraes E, Chintakunta SS, Boominathan SA (2025) How effective are llms for data science coding? a controlled experiment. In: 2025 IEEE/ACM 22nd international conference on mining software repositories (MSR), Ottawa, ON, Canada, pp 211–222. https://doi.org/10.1109/MSR66628.2025.00041

OpenAI (2021) Openai codex. https://openai.com/codex/, last accessed: October 2025

OpenAI (2024) Chatgpt. https://openai.com/chatgpt, model: GPT-4, last accessed: Obtober 2025

Ostrand TJ, Balcer MJ (1988) The category-partition method for specifying and generating fuctional tests. Commun ACM 31(6):676–686. https://doi.org/10.1145/62959.62964

Ouyang S, Zhang JM, Harman M, Wang M (2025) An empirical study of the non-determinism of chatgpt in code generation. ACM Trans Softw Eng Methodol 34(2). https://doi.org/10.1145/3697010

Palla D, Slaby A (2025) Evaluation of generative ai models in python code generation: a comparative study. IEEE Access 13:65334–65347. https://doi.org/10.1109/ACCESS.2025.3560244

Peng S, Kalliamvakou E, Cihon P, Demirer M (2023) The impact of ai on developer productivity: Evidence from github copilot. https://doi.org/10.48550/arXiv.2302.06590,

Piya S, Sullivan A (2024) Llm4tdd: best practices for test driven development using large language models. In: Proceedings of the 1st international workshop on large language models for code, Association for Computing Machinery, New York, NY, USA, LLM4Code '24, p 14–21. https://doi.org/10.1145/3643795.3648382

Piya S, Samadi A, Sullivan A (2025) Is more or less automation better? An investigation into the llm4tdd process. In: 2025 IEEE/ACM international workshop on large language models for code (LLM4Code), Ottawa, ON, Canada, pp 161–168. https://doi.org/10.1109/LLM4Code66737.2025.00025

Richards TB (2023) Auto-gpt: an autonomous gpt-4 experiment. https://github.com/Torantulino/Auto-GPT, last accessed: October 2025

Romano S, Sferratore FP, Scanniello G (2025) Further evidence on a controversial topic about human-based experiments: Professionals vs. students. https://doi.org/10.48550/arXiv.2506.11597,

Salman I, Misirli AT, Juristo N (2015) Are students representatives of professionals in software engineering experiments? In: 2015 IEEE/ACM 37th IEEE international conference on software engineering, Florence, Italy, pp 666–676. https://doi.org/10.1109/ICSE.2015.82

Sapkota R, Roumeliotis KI, Karkee M (2025) Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai. https://doi.org/10.48550/arXiv.2505.19443,

Shin J, Tang C, Mohati T, Nayebi M, Wang S, Hemmati H (2025) Prompt engineering or fine-tuning: an empirical assessment of llms for code. In: 2025 IEEE/ACM 22nd international conference on mining software repositories (MSR), IEEE Computer Society, Ottawa, ON, Canada, pp 490–502. https://doi.org/10.1109/MSR66628.2025.00082

Siegmund J, Kästner C, Liebig J, Apel S, Hanenberg S (2014) Measuring and modeling programming experience. Empir Softw Engg 19(5):1299–1334. https://doi.org/10.1007/s10664-013-9286-4

Talebirad Y, Nadiri A (2023) Multi-agent collaboration: Harnessing the power of intelligent llm agents. https://doi.org/10.48550/arXiv.2306.03314

Tholander J, Jonsson M (2026) Vibe coding entanglements – repositioning boundaries of intention, authorship, and responsibility in programming with generative ai. In: Proceedings of the 2026 CHI conference on human factors in computing systems, Association for Computing Machinery, New York, NY, USA, CHI '26. https://doi.org/10.1145/3772318.3791847

Tie J, Yao B, Li T, Ahmed SI, Wang D, Zhou S (2024) Llms are imperfect, then what? An empirical study on llm failures in software engineering. https://doi.org/10.48550/arXiv.2411.09916,

Tran HKV, Unterkalmsteiner M, Börstler J, bin Ali N, (2021) Assessing test artifact quality—a tertiary study. Inf Softw Technol 139:106620. https://doi.org/10.1016/j.infsof.2021.106620, https://www.sciencedirect.com/science/article/pii/S0950584921000938

Tran HKV, Ali Nb, Unterkalmsteiner M, Börstler J, Chatzipetrou P (2025) Quality attributes of test cases and test suites – importance & challenges from practitioners' perspectives. Software Quality Journal 33(1). https://doi.org/10.1007/s11219-024-09698-w

Truong J, Nguyen V, Nguyen TT (2026) An overview of evaluation and enhancement methods for code generation by large language models. Inf Softw Technol 197:108185. https://doi.org/10.1016/j.infsof.2026.108185, https://www.sciencedirect.com/science/article/pii/S0950584926001746

Tufano M, Drain D, Svyatkovskiy A, Sundaresan N (2022) Generating accurate assert statements for unit test cases using pretrained transformers. In: Proceedings of the 3rd ACM/IEEE international conference on automation of software test, ACM, Pittsburgh, Pennsylvania, USA, pp 54–64. https://doi.org/10.1145/3524481.3527220

Ulfsnes R, Moe NB, Stray V, Skarpen M (2024) Transforming software development with generative AI: empirical insights on collaboration and workflow, Springer Nature Switzerland, Brug, Switzerland, pp 219–234. https://doi.org/10.1007/978-3-031-55642-5_10

Vaidya J, Asif H (2023) A critical look at ai-generate software: coding with the new ai tools is both irresistible and dangerous. IEEE Spectr 60(7):34–39. https://doi.org/10.1109/MSPEC.2023.10177044

Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Lu, Polosukhin I (2017) Attention is all you need. In: Guyon I, Luxburg UV, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R (eds) Advances in neural information processing systems, Curran Associates, Inc., vol 30

Vecchio MD, Fabbri D, Marchisa E (2024) Copilots for coding. https://www.eng.it/content/dam/eng-portal/resources/documents/instant-paper/cross/copilots-for-coding/ENG24_IP_GenAI-Coding_EN.pdf, last accessed: September 2025

Wang J, Huang Y, Chen C, Liu Z, Wang S, Wang Q (2024) Software testing with large language models: survey, landscape, and vision. IEEE Trans Softw Eng 50(4):911–936. https://doi.org/10.1109/TSE.2024.3368208

Wasserstein RL, Schirm AL, Lazar NA (2019) Moving to a world beyond "p < 0.05". Am Stat 73(sup1):1–19. https://doi.org/10.1080/00031305.2019.1583913

Weyssow M, Zhou X, Kim K, Lo D, Sahraoui H (2025) Exploring parameter-efficient fine-tuning techniques for code generation with large language models. ACM Transactions on Software Engineering and Methodology 1–24. https://doi.org/10.1145/3714461,

White J, Fu Q, Hays S, Sandborn M, Olea C, Gilbert H, Elnashar A, Spencer-Smith J, Schmidt DC (2023) A prompt pattern catalog to enhance prompt engineering with chatgpt. In: Proceedings of the 30th conference on pattern languages of programs, The Hillside Group, USA, PLoP '23. https://dl.acm.org/doi/10.5555/3721041.3721046

White J, Hays S, Fu Q, Spencer-Smith J, Schmidt DC (2024) ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design, Springer Nature Switzerland, Cham, pp 71–108. https://doi.org/10.1007/978-3-031-55642-5_4

Wohlin C, Runeson P, Höst M, Ohlsson MC, Regnell B, Wesslén A (2012) Empirical strategies. Springer Nature Link pp 9–36. https://doi.org/10.1007/978-3-642-29044-2_2

Yang J, Jin H, Tang R, Han X, Feng Q, Jiang H, Zhong S, Yin B, Hu X (2024) Harnessing the power of llms in practice: A survey on chatgpt and beyond. ACM Trans Knowl Discov Data 18(6). https://doi.org/10.1145/3649506

Zhu H, Hall PAV, May JHR (1997) Software unit test coverage and adequacy. ACM Comput Surv 29(4):366–427. https://doi.org/10.1145/267580.267590

Publisher"s Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Moritz Mock is a third-year PhD student in Advanced Systems Engineering (ASE) at the Free University of Bozen-Bolzano. He holds an M.Sc. in Software Engineering for Information Systems from the same university. His research focuses on mining open-source software to detect vulnerabilities using static analysis and machine learning techniques. He has published at ASE, MSR, and ICPC, and has served as a reviewer for MSR, ESEM, ICSME, IST, JSS, and TOSEM. He is a member of Prof. Barbara Russo's research group.



Barbara Russo is a Full Professor of Computer Science at the Faculty of Engineering at the Free University of Bozen-Bolzano. She has served as Vice Dean for Research at the Faculty of Computer Science and Technologies and has coordinated scientific programs for international symposia and doctoral schools. Barbara Russo is an Associate Editor for the International Journal of Information and Software Technology (Elsevier) and acts as a reviewer for several high-level journals and conferences in software engineering (e.g., ICSE, MSR, EMSE, FSE) journals (TSE, TOSEM, ESE, ASE) in Software Engineering. She has published 200 articles in international mathematics and computer science conferences and journals, which have received over 4000 citations (h index=34) . Her expertise lies in software development and maintenance aimed at ensuring systems with high standards of reliability, scalability, performance, and security. In the last four years, prof. Russo' research specialises in data extraction using Deep Learning techniques to reconstruct system and user behaviour and detect anomalies and code vulnerabilities.

Authors and Affiliations

Moritz Mock[1] · **Barbara Russo**[1]

✉ Moritz Mock
momock@unibz.it

Barbara Russo
brusso@unibz.it

[1] Faculty of Engineering, Free University of Bozen-Bolzano, Bolzano, Italy